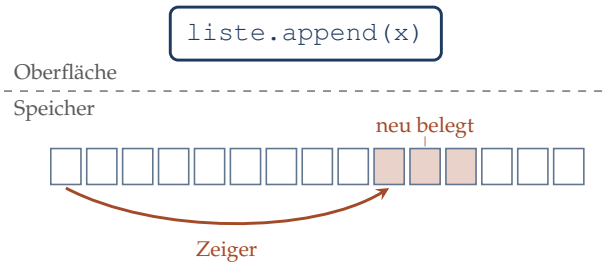
Informatik von Grund auf — Band 21

Was darunter liegt

Programmieren in C



Was Python verbirgt und was es kostet

Jeder Schritt erklärt. Mit vollständig entwickelten Programmen.

Stand der Forschung: August 2026

Informatik von Grund auf — Band 21
Was darunter liegt · Programmieren in C

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 21: Was darunter liegt*. Fassung 1.0.0.
`https://github.com/juergenmaier2026/informatik-von-grund-auf`

Satz: LaTeX mit Palatino. Fassung 1.0.0, Oktober 2026. 132 Seiten.

Inhaltsverzeichnis

Vorwort 1

I Die Maschine wird sichtbar 7

1 Was verborgen war 9

2 Speicher 15

3 Zeiger und Besitz 23

II Der erste Entwurf in C 31

4 Zeichenketten 33

5 Fehler ohne Ausnahmen 39

6 Datenstrukturen von Hand 45

III Die Fallen 53

7 Undefiniertes Verhalten 55

8 Ganzzahlen und Gleitkomma 63

9 Der Übersetzer 69

IV Das Handwerk 75

10 Werkzeuge 77

11 Module und Schnittstellen 83

12 Fehlersuche in C 89

V Das Urteil **97**

13 Der zweite Entwurf in C **99**

14 Wann C und wann nicht **105**

15 Der Stand im Jahr 2026 **111**

VI Anhang **117**

A Regeln und Kenngrößen **119**

B Landkarte und Glossar **123**

C Kommentierte Quellen **125**

Vorwort: Die Rechnung wird sichtbar

In Python schreibt man `liste.append(x)` und denkt nicht weiter. *In C schreibt man dieselbe Operation in zwölf Zeilen und trifft dabei sechs Entscheidungen, die vorher jemand anders getroffen hat. Dieser Band handelt davon, welche das sind — und was sie kosten.*

Was eine Zeile verbirgt

Wer besitzt den Speicher?	Wer gibt ihn frei, und wann?
Wie viel wurde belegt?	Und was geschieht, wenn es nicht reicht?
Wird umkopiert?	**Bleiben alte Zeiger gültig?**
Was bei Fehlschlag?	Was, wenn kein Speicher mehr da ist?
Wer prüft die Grenzen?	**Niemand**
Ist es nebenläufig sicher?	

Sechs Fragen, die Python beantwortet hat, bevor man sie stellen konnte. In C muss man sie alle beantworten — und die fünfte lautet immer gleich: niemand. Das ist der Kern dieses Bandes.

Der Preis in beide Richtungen.

	Python	C
Entscheidungen	**getroffen**	**selbst zu treffen**
Fehlerklassen	ausgeschlossen	**alle offen**
Vorhersagbarkeit	begrenzt	**vollständig**
Zeilen je Aufgabe	1	≈ 10–30
Kontrolle	gering	vollständig

Die dritte Zeile ist der Grund, warum es C noch gibt. Wenn eine Bremse in fünf Millisekunden reagieren muss, ist eine automatische Speicherbereinigung, die gelegentlich zwanzig Millisekunden anhält, nicht „etwas langsamer“ — sie ist unbrauchbar.

Was dieser Band nicht ist

Inhalt	Enthalten?	
C-Syntax erklärt	**nein**	**steht in [14]**
Vollständige Bibliotheksreferenz	nein	
Was darunter geschieht	**ja**	
Welche Entscheidungen anfallen	ja	
Wo die Fallen liegen	**ja**	**Teil III**
Wann C die richtige Wahl ist	ja	

Wer C lernen will, nimmt Kernighan und Ritchie — zweihundert Seiten, seit 1988 unübertroffen. Dieser Band setzt voraus, dass Sie die Sprache lesen können, und behandelt, was man damit anstellt und wovor man sich hüten muss.

Die Fortsetzung des Vorhabens

Dasselbe Modell wie in Band 20 — Befunde, Verläufe, Medikation — nun eine Ebene tiefer.

Warum derselbe Fall	Man sieht den Unterschied der Sprachen, nicht den der Aufgaben
Was neu hinzukommt	**Ein Messdatenerfasser, der in Echtzeit arbeitet**
Warum der passt	Harte Zeitschranken, begrenzter Speicher — **genau wo C hingehört**

Die zweite Zeile ist die Erweiterung: Ein Gerät am Patientenbett, das Messwerte aufnimmt und weiterreicht. Wenige Kilobyte Speicher, feste Zeitschranken, kein Betriebssystem im üblichen Sinn — die Umgebung, in der die Fragen dieses Bandes nicht akademisch sind.

Dieselben Fragen, andere Antworten.

Frage	In Python	In C
Ergebnistyp mit Fällen	Klassen	**Union mit Kennzeichen**
Unveränderlichkeit	`frozen`	**Vereinbarung**
Fehler melden	Ausnahme	**Rückgabewert**
Ungültiger Wert	`None`	Nullzeiger — gefährlich
Zeichenkette	`str`	**Zeiger und Länge**

Fünf Entwurfsentscheidungen aus Band 20, hier neu zu treffen. Die zweite ist die unbequemste: C kennt keine erzwungene Unveränderlichkeit — `const` ist ein Versprechen, das man umgehen kann. Was in Python die Sprache sicherstellt, ist hier Disziplin.

Was dieses Buch will

Erstens: die Kosten sichtbar machen.

Zu jeder Bequemlichkeit aus Band 20 gehört eine Rechnung. Dieser Band stellt sie auf: wie viele Zeilen, wie viele Entscheidungen, wie viele mögliche Fehler — und was man dafür bekommt.

Zweitens: das undefinierte Verhalten ernst nehmen.

Es ist die Eigenheit von C, die am meisten Schaden anrichtet und am wenigsten verstanden wird. Kapitel 7 zeigt, warum ein Programm mit undefiniertem Verhalten nicht „manchmal falsch“ ist, sondern gar keine Bedeutung mehr hat.

Drittens: sagen, wann man C nicht nehmen sollte.

Ein Buch über eine Sprache, das keine Grenzen nennt, ist Werbung. Kapitel 14 nennt sie — und die Liste ist länger, als C-Programmierer gern hören.

Drei Warnungen

Erstens: C ist keine Hochsprache für die Maschine.

> *Der Satz „C ist portabler Assembler“ war 1978 richtig und ist heute falsch. Zwischen dem C-Modell und einem heutigen Prozessor liegen mehrere Übersetzungsstufen — Kapitel 9 führt das aus.*

Zweitens: Der Übersetzer ist Ihr Gegner, wenn Sie die Regeln brechen.
Er darf annehmen, dass undefiniertes Verhalten nicht vorkommt — und daraus Schlüsse ziehen, die ganze Prüfungen entfernen. Wer die Regeln einhält, hat einen Verbündeten; wer sie bricht, einen Optimierer gegen sich.
Drittens: Vorsicht ersetzt keine Werkzeuge.

Die Fehlerklassen dieses Bandes werden nicht durch Sorgfalt vermieden — die Zahl der Gelegenheiten schlägt die Sorgfalt, wie Band 19 gezeigt hat. Sie werden durch Werkzeuge gefunden, und Kapitel 10 behandelt sie.

Zum Aufbau

- **Teil I** (Kap. 1–3): Die Maschine wird sichtbar. **Was verborgen war**, Speicher, **Zeiger und Besitz**.
- **Teil II** (Kap. 4–6): Der erste Entwurf in C. Zeichenketten, Fehler ohne Ausnahmen, Datenstrukturen.
- **Teil III** (Kap. 7–9): Die Fallen. **Undefiniertes Verhalten**, Zahlen, der Übersetzer.
- **Teil IV** (Kap. 10–12): Das Handwerk. **Werkzeuge**, Module, Fehlersuche.
- **Teil V** (Kap. 13–15): Das Urteil. Der zweite Entwurf, **wann C**, der Stand 2026.

Eine fehlende Längenprüfung

2014 wurde in einer weltweit eingesetzten Verschlüsselungsbibliothek ein Fehler gefunden, der zwei Jahre lang unbemerkt geblieben war.

Der Vorgang	Ein Gegenüber sendet einen Text und dessen Länge
Die Antwort	Derselbe Text wird zurückgeschickt
Die Lücke	**Die gesendete Länge wurde nicht gegen die tatsächliche geprüft**
Die Folge	Bis zu 64 Kilobyte fremden Speichers wurden mitgeliefert

Der Fehler ist eine einzige fehlende Zeile. Er war möglich, weil in C eine Länge und ein Zeiger zwei getrennte Angaben sind und nichts ihren Zusammenhang erzwingt — genau die Trennung, die Kapitel 4 behandelt.

Was der Fall über C sagt — und was nicht.

Schluss	**Trägt?**	
„C ist unbrauchbar“	nein	**zu einfach**
„Die Entwickler waren nachlässig“	**nein**	Band 19, Kap. 4
Die Sprache lässt den Fehler zu	**ja**	
Werkzeuge hätten ihn gefunden	ja	Kap. 10

Die letzte Zeile ist die praktische Lehre: Ein Speicherprüfer hätte den Zugriff über die Puffergrenze bei der ersten Prüfung gemeldet. Das Werkzeug gab es, es lief nur nicht.

C gibt Ihnen die Kontrolle über alles.
Auch über die Fehler,
die andere Sprachen für Sie ausschließen.

— der Leitgedanke dieses Bandes.
Die Frage ist nie, ob C mächtig genug ist.
Sie ist, ob Sie die Kontrolle brauchen, die Sie bezahlen.

Part I

Die Maschine wird sichtbar

1 Was verborgen war

Eine Zeile, aufgefaltet

In Python.

```
befunde.append(neuer)
```

In C, vollständig.

```
int befund_anfuegen(Liste *l, const Befund *b) {
if (l->anzahl == l->kapazitaet) {
size_t neu = l->kapazitaet ?  l->kapazitaet * 2 :  8;
if (neu > SIZE_MAX / sizeof *l->daten)
return -ENOMEM;
Befund *p = realloc(l->daten, neu * sizeof *l->daten);
if (!p)
return -ENOMEM;
l->daten = p;
l->kapazitaet = neu;
}
l->daten[l->anzahl++] = *b;
return 0;
}
```

Vierzehn Zeilen für eine. Sie sind nicht umständlich — jede beantwortet eine der sechs Fragen aus dem Vorwort. Die vierte Zeile prüft den Überlauf der Größenberechnung; ohne sie hätte man den Fehler aus Band 19, Kapitel 4.

Die sechs Entscheidungen, sichtbar gemacht.

Frage	Hier beantwortet	Alternative
Wachstumsfaktor	**verdoppeln**	1,5×, fest
Anfangsgröße	8	aufgabenabhängig
Überlauf	**geprüft**	meist vergessen
Fehlschlag	**Rückgabewert**	abbrechen
Besitz	**Liste besitzt**	Aufrufer besitzt
Kopie	**Wert kopiert**	Zeiger übernommen

Die letzte Zeile ist die folgenreichste und im Code nur an einem Sternchen erkennbar: `*b` *kopiert den Befund. Hätte man den Zeiger gespeichert, wäre die Liste von der Lebensdauer des Originals abhängig — und genau daraus entstehen die Fehler aus Kapitel 3.*

Warum verdoppeln

Wachstum	Umlagerungen	Kopierte Elemente	Je Einfügung
$+1$ fest	n	$\mathbf{n^2/2}$	$\mathcal{O}(n)$
$+100$ fest	$n/100$	$n^2/200$	$\mathcal{O}(n)$
$\times 2$	$\mathbf{\log_2 n}$	$\mathbf{\approx 2n}$	$\mathcal{O}(\mathbf{1})$
$\times 1{,}5$	$\log_{1,5} n$	$\approx 3n$	$\mathcal{O}(1)$

Nur geometrisches Wachstum gibt amortisiert konstante Kosten. Bei festem Zuwachs kopiert man quadratisch viel — bei einer Million Elementen und Schrittweite hundert sind das fünf Milliarden Kopieroperationen statt zwei Millionen.

Warum manche 1,5 nehmen.

	Faktor 2	**Faktor 1,5**
Kopieraufwand	geringer	etwas höher
Speicherverschnitt	bis 50 %	bis 33 %
Freigewordenes wiederverwendbar	**nie**	**ab dem vierten Mal**

Die letzte Zeile ist das eigentliche Argument: Bei Verdopplung ist die Summe aller bisherigen Blöcke immer kleiner als der nächste — der freigewordene Speicher passt nie. Bei 1,5 passt er irgendwann. Ob das zählt, hängt vom Speicherverwalter ab.

Was Python an derselben Stelle tut

Entscheidung	Python	
Wachstum	$\approx 1{,}125\times$	**plus Konstante**
Elemente	**Zeiger**	nicht die Werte
Überlauf	geprüft	
Fehlschlag	Ausnahme	

Die zweite Zeile ist der wichtige Unterschied: Eine Python-Liste speichert Verweise, keine Werte. Deshalb kostet das Umlagern immer gleich viel, unabhängig von der Objektgröße — und deshalb braucht jedes Element zusätzlich seinen eigenen Speicherblock.

Aufgabe 1.1.

(a) Rechnen Sie die Speicherkosten beider Darstellungen aus.
(b) Bestimmen Sie, was ein Zugriff kostet.
(c) Untersuchen Sie, wo die Zeit tatsächlich hingeht.

Lösung Schritt für Schritt.

(a) Speicherkosten.
Der Fall. Eine Million Befunde, je vier Zahlenfelder.

Posten	C	Python
Nutzdaten	32 B	32 B
Objektkopf	0	$\approx$ 56 B
Zeiger im Feld	0	8 B
Verwaltung je Block	0	$\approx$ 16 B
Je Element	**32 B**	$\approx$ **112 B**
Bei 10^6	**32 MB**	$\approx$ **112 MB**

Faktor dreieinhalb — und das ist die günstige Rechnung, weil sie eine Datenklasse mit festen Feldern annimmt. Bei gewöhnlichen Objekten mit Wörterbuch kommt weiterer Aufwand hinzu.

Die zweite Wirkung.

	C	Python	
Zusammenhängend	**ja**	**nein**	
Elemente je Zwischenspeicherzeile	**2**	≈ 0	
Durchlauf 10^6	$\approx$ 1,2 ms	$\approx$ 48 ms	40×

Der Faktor vierzig beim Durchlauf stammt nicht aus der Sprachgeschwindigkeit, sondern aus der Anordnung im Speicher: In C liegen die Werte hintereinander, in Python liegen Zeiger hintereinander und die Werte verstreut. Kapitel 2 rechnet das aus.

(b) Was ein Zugriff kostet.

Vorgang	Dauer	
Register	$<$ 1 Takt	
Zwischenspeicher L1	$\approx$ 4 Takte	$\approx$ 1 **ns**
L2	$\approx$ 14 Takte	
L3	$\approx$ 50 Takte	
Hauptspeicher	$\approx$ 250 Takte	$\approx$ 80 **ns**

> *Zwischen dem schnellsten und dem langsamsten Zugriff liegt Faktor zweihundertfünfzig — innerhalb desselben Prozessors. Ein Programm, das die Zwischenspeicher trifft, ist um Größenordnungen schneller als eines, das sie verfehlt, bei identischem Code.*

Das ist die Erklärung für den Faktor vierzig oben und zugleich die wichtigste Einsicht dieses Bandes: In C bestimmt die Anordnung *der Daten die Geschwindigkeit stärker als das Verfahren. Wirths Rangordnung aus Band 20 gilt hier doppelt.*

(c) Wo die Zeit hingeht.
Der Messdatenerfasser. 500 Messwerte je Sekunde, Zeitschranke 2 ms je Wert.

Schritt	C	Python	Schranke
Wert einlesen	12 µs	95 µs	ok
Umrechnen	3 µs	41 µs	ok
Prüfen	2 µs	28 µs	ok
Anfügen	1 µs	9 µs	ok
Summe	**18** µs	173 µs	beide ok
Bereinigungspause	**0**	**0–14 ms**	**verletzt**

Die Summenzeile spricht für Python: hundertdreiundsiebzig Mikrosekunden gegen eine Schranke von zweitausend — reichlich Luft. Die letzte Zeile entscheidet: Eine gelegentliche Pause der Speicherbereinigung von vierzehn Millisekunden verletzt die Schranke um das Siebenfache.

Das ist der ehrliche Grund für C an dieser Stelle — nicht die mittlere Geschwindigkeit, sondern die Vorhersagbarkeit. *Für eine Auswertung, die abends läuft, wäre die Rechnung umgekehrt ausgegangen.*

Der Aufwand fällt nicht nur beim Schreiben an

Größe	Python	C	Faktor
Zeilen	1	14	14
Prüffälle nötig	0	**6**	
Mögliche Fehler	≈ 0	**5**	
Durchsichtszeit	2 s	≈ 90 s	45

Die dritte Zeile ist die teuerste: Überlauf, Fehlschlag der Belegung, veralteter Zeiger beim Aufrufer, Kopie statt Verweis, Größenberechnung mit falschem Typ. Fünf Gelegenheiten, wo vorher keine war — und Band 19 hat gezeigt, dass die Zahl der Gelegenheiten die Sorgfalt schlägt.

Das ist kein Argument gegen C. Es ist die Rechnung, die man aufmachen muss, bevor man sich dafür entscheidet — und Kapitel 14 macht sie vollständig auf.

Chisnall[4] argumentiert, dass C die Maschine nicht mehr abbildet, die es abzubilden vorgibt; Bryant und O'Hallaron[3] erklären, was tatsächlich unter dem Code geschieht; Ritchie[22] beschreibt, aus welchen Zwängen die Sprache entstand.

2 Speicher

Drei Orte

	Statisch	Keller	Halde
Lebensdauer	**Programm**	**Funktion**	**bis Freigabe**
Größe	fest	begrenzt	frei
Kosten	0	≈ 0	50–200 ns
Fehlschlag	Überlauf	Überlauf	meldbar
Freigeben	—	automatisch	**von Hand**

Die dritte Zeile wird unterschätzt: Eine Belegung auf der Halde kostet so viel wie ein Hauptspeicherzugriff. Wer in einer Schleife hunderttausendmal belegt und freigibt, verbringt mehr Zeit in der Speicherverwaltung als in der eigentlichen Arbeit.

Die Kellergrenze.

Umgebung	Kellergröße	
Übliches System	8 MB	je Faden meist ≈ 1 MB
Eingebettet	**4–64 KB**	**Messdaten-erfasser**
Kernbereich	8–16 KB	

Bei vier Kilobyte Keller ist ein lokales Feld von tausend Befunden unmöglich — es wären zweiunddreißig Kilobyte. Der Übersetzer meldet das nicht; das Programm läuft, bis es den Keller überschreibt und irgendwo anders abstürzt.

Warum Anordnung wichtiger ist als Verfahren

Anordnung	Durchlauf 10^6	
Feld von Werten	**1,2 ms**	Zeile voll genutzt
Feld von Zeigern, geordnet	4,8 ms	
Feld von Zeigern, verstreut	**47 ms**	jede Zeile neu
Verkettete Liste	**83 ms**	**zusätzlich unvorhersagbar**

Faktor siebzig zwischen der ersten und der letzten Zeile — bei identischer Zahl von Operationen. Die verkettete Liste ist im Lehrbuch gleichwertig und in der Praxis um zwei Größenordnungen langsamer, weil der Prozessor nicht vorhersagen kann, welche Adresse als Nächstes gebraucht wird.

Die Zwischenspeicherzeile

Die Einheit	Der Prozessor holt nie ein Byte, sondern 64 **Byte am Stück**
Die Folge	Wer ein Byte braucht, bezahlt für vierundsechzig
Der Gewinn	**Wer die anderen dreiundsechzig auch braucht, bekommt sie umsonst**

Daraus folgt die wichtigste Entwurfsregel dieses Kapitels: Was zusammen verarbeitet wird, sollte zusammen liegen. Nicht was logisch zusammengehört — was gemeinsam gelesen *wird.*

Der Unterschied an unserem Fall.

```
/* Anordnung A: Feld von Verbunden */
struct Befund { double wert; time_t t; int code;
char labor[32]; };
struct Befund b[N];

/* Anordnung B: Verbund von Feldern */
struct Befunde { double wert[N]; time_t t[N];
int code[N]; char labor[N][32]; };
```

Aufgabe	A	B	
Mittelwert aller Werte	9,4 ms	**1,1 ms**	8,5×
Einen Befund ausgeben	**0,1** µs	0,4 µs	4×

Keine Anordnung ist besser — sie sind für verschiedene Zugriffsmuster gut. A ist richtig, wenn man ganze Befunde bearbeitet; B, wenn man Spalten auswertet. Die Wahl folgt aus der Aufgabe, nicht aus dem Geschmack.

Die Auffüllung im Verbund

```
struct A { char c; double d; int i; };  /* 24 Byte
*/
struct B { double d; int i; char c; };  /* 16 Byte
*/
```

	Nutzdaten	Tatsächlich	Verschnitt
`A`	13 B	**24** B	**46** %
`B`	13 B	16 B	19 %

Dieselben Felder, andere Reihenfolge, fünfzig Prozent mehr Speicher. Der Grund ist die Ausrichtung: Ein `double` muss an einer durch acht teilbaren Adresse liegen, und der Übersetzer füllt dazwischen auf. Die Regel lautet: absteigend nach Größe anordnen.

Bei einer Million Befunden sind das acht Megabyte Unterschied — und die wirken sich nicht nur auf den Speicher aus, sondern über die Zwischenspeicherzeilen auch auf die Geschwindigkeit.

Aufgabe 2.1.

(a) Wählen Sie die Anordnung nach dem Zugriffsmuster.
(b) Rechnen Sie die Kosten der Haldenbelegung aus.
(c) Bestimmen Sie, wann ein Speicherpool lohnt.

Lösung Schritt für Schritt.

(a) Anordnung nach Zugriffsmuster.
Die Zugriffe im Messdatenerfasser.

Vorgang	Häufigkeit	Felder	Besser
Neuen Wert anfügen	500/s	**alle**	**A**
Gleitmittel über 60 s	1/s	**nur Wert**	**B**
Grenzprüfung	500/s	Wert, Code	B
Block übertragen	0,1/s	alle	A

Die Häufigkeiten entscheiden: Zweimal 500 *je Sekunde gegen einmal* 1 *und einmal* 0,1*. Die dritte Zeile spricht für B, die erste für A — und die erste ist der häufigste Vorgang mit der härtesten Zeitschranke.*

Der Kompromiss.

```
/* heiss:  zusammen gelesen und geschrieben */
struct Kern { double wert; uint32_t t; uint16_t code;
};
struct Kern kern[N];

/* kalt:  selten gebraucht, getrennt gehalten */
struct Zusatz { char labor[32]; uint8_t qualitaet; };
struct Zusatz zusatz[N];
```

	A	B	Geteilt
Anfügen	**1,0** µs	2,1 µs	**1,2** µs
Gleitmittel	9,4 ms	**1,1** ms	**1,4** ms
Kern je Element	48 B	48 B	**16 B**

Die Aufteilung in heiße und kalte Felder bringt fast die Vorteile beider Anordnungen. Der Kern schrumpft auf sechzehn Byte — vier Elemente je Zwischenspeicherzeile statt eineinhalb.

(b) Kosten der Haldenbelegung.

Vorgang	Dauer	
`malloc`, klein	≈ 60 ns	aus Freiliste
`malloc`, groß	≈ 200 ns	
`malloc` mit Systemaufruf	≈ 2 µs	**neue Seite**
`free`	≈ 40 ns	
Kellervariable	≈ **0**	**ein Register**

Eine Belegung und Freigabe kostet zusammen etwa hundert Nanosekunden — so viel wie ein Hauptspeicherzugriff. In einer Schleife über eine Million Elemente sind das hundert Millisekunden, die nichts mit der Aufgabe zu tun haben.

Die Vermeidung.

Vorgehen	Belegungen	Dauer
Je Element eine	10^6	**102 ms**
Einmal für alle	**1**	**0,2 ms**
Puffer wiederverwenden	1	0,2 ms

Faktor fünfhundert allein durch das Vermeiden von Belegungen. Das ist die erste Optimierung, die man in C vornimmt — und sie ist meist eine Entwurfsentscheidung, keine nachträgliche Verbesserung.

(c) Wann ein Speicherpool lohnt.

Lage	Pool?	
Viele gleich große Objekte	**ja**	**Freiliste genügt**
Alle zugleich freigegeben	**ja**	**Arena**
Feste Zeitschranken	ja	vorhersagbar
Wenige, langlebige Objekte	nein	Aufwand ohne Ertrag
Unterschiedliche Größen	nein	Verschnitt

Die zweite Zeile beschreibt das mächtigste Muster: eine Arena. Man belegt einen großen Block, gibt daraus Stücke aus, ohne je einzeln freizugeben — und wirft am Ende den ganzen Block weg. Freigabe kostet dann eine Operation statt einer Million.

Die Arena am Fall.

```
typedef struct { char *anfang, *frei, *ende; } Arena;

void *arena_hole(Arena *a, size_t n, size_t ausr) {
uintptr_t p = (uintptr_t)a->frei;
p = (p + ausr - 1) & ~(ausr - 1);
if ((char *)(p + n) > a->ende) return NULL;
a->frei = (char *)(p + n);
return (void *)p;
}
```

	`malloc`	**Arena**	
Belegung	60 ns	**3 ns**	20×
Freigabe je Objekt	40 ns	**0**	
Vorhersagbar	nein	**ja**	
Leck möglich	ja	**nein**	
Einzelfreigabe	ja	nein	

Die vierte Zeile ist der eigentliche Gewinn und wird meist übersehen: In einer Arena kann es kein Speicherleck geben, weil niemand einzeln freigibt. Eine ganze Fehlerklasse verschwindet — dieselbe Bauweise wie in Band 20, Kapitel 6.

Der Preis steht in der letzten Zeile und ist echt: Wo Objekte verschieden lange leben, taugt die Arena nicht. Bei einem Messdatenerfasser, der je Zyklus alles verwirft, passt sie genau.

Speicher ist nicht flach

Annahme	Zutreffend?	
Jeder Zugriff kostet gleich	**nein**	**Faktor** 250
Reihenfolge egal	nein	Vorabholung
Nur die Zahl der Zugriffe zählt	**nein**	
Zwei Programme, gleiche Operationen, gleiche Zeit	nein	**Faktor** 70 **möglich**

Die Aufwandsanalyse aus Band 1 zählt Operationen und nimmt an, dass jede gleich viel kostet. Diese Annahme war 1970 richtig. Heute liegt zwischen dem schnellsten und dem langsamsten Zugriff Faktor zweihundertfünfzig — und damit kann ein Verfahren mit mehr Operationen schneller sein.

Das ist kein Einwand gegen die Aufwandsanalyse — sie sagt weiterhin, wie es sich bei wachsendem n verhält. Sie sagt nur nichts mehr über die Konstante, und die entscheidet in der Praxis öfter, als einem lieb ist.

Drepper[6] erklärt die Speicherhierarchie so gründlich wie kein zweiter Text; Bryant und O'Hallaron[3] ordnen sie in das Gesamtbild ein; Wirth[28] stellte die Rangordnung von Datenstruktur und Verfahren auf, die hier erneut greift.

3 Zeiger und Besitz

Was ein Zeiger nicht sagt

Frage	Im Typ?	
Worauf zeigt er?	ja	`Befund *`
Zeigt er auf etwas?	**nein**	**Nullzeiger**
Auf wie viele?	nein	**eins oder tausend**
Wie lange gültig?	**nein**	Lebensdauer
Wer gibt frei?	**nein**	**Besitz**
Darf man schreiben?	teils	`const` — umgehbar

Sechs Fragen, fünf unbeantwortet. Ein Zeigertyp trägt genau eine Information — die Größe dessen, worauf er zeigt — und lässt alles offen, was für die Richtigkeit zählt. Diese Lücke ist die Quelle der meisten Fehler in C.

Die Fehlerklassen daraus.

Fehler	Aus welcher Lücke	Wirkung
Nullzeigerzugriff	**zeigt er auf etwas**	Absturz
Grenzüber-schreitung	**auf wie viele**	**stiller Datenschaden**
Zugriff nach Freigabe	**wie lange gültig**	fremde Daten
Doppelte Freigabe	**wer gibt frei**	Verwaltung kaputt
Speicherleck	wer gibt frei	wächst

Die zweite Zeile ist die gefährlichste, weil sie lautlos ist: Ein Zugriff hinter das Feldende stürzt nicht ab, er überschreibt etwas anderes. Der Fehler zeigt sich Stunden später an einer Stelle, die nichts damit zu tun hat.

Besitz als Vereinbarung

C kennt keinen Besitzbegriff. Man muss ihn selbst einführen und einhalten.

Regel	Bedeutet	Erkennbar an
Aufrufer besitzt	**gibt selbst frei**	`const T *`
Aufgerufener übernimmt	**gibt frei**	**Name: `..._uebernimm`**
Geliehen	nur während des Aufrufs	**gefährlich**
Geteilt	Zählung nötig	aufwendig

Die dritte Zeile ist die häufigste und die riskanteste: Ein geliehener Zeiger darf den Aufruf nicht überleben. Nichts erzwingt das — wenn der Aufgerufene ihn speichert, entsteht ein Zeiger auf Speicher, der später verschwindet.

Die drei Mittel, die es gibt.

Mittel	Wirkung	
Namensgebung	Hinweis	**kostenlos, unverbindlich**
Kommentar an der Signatur	Hinweis	veraltet
Werkzeuge	**prüfen**	**Kapitel 10**

Nur das dritte Mittel prüft tatsächlich. Die ersten beiden sind Konventionen, und Konventionen verlieren gegen die Zahl der Gelegenheiten. Das ist derselbe Befund wie in Band 19, Kapitel 5.

Der Nullzeiger

TONY HOARE führte ihn 1965 ein und nannte ihn später seinen Milliardenfehler.

Der Gedanke	Ein Zeiger, der auf nichts zeigt, als Sonderwert
Der Fehler	**Jeder Zeiger kann null sein, ohne dass der Typ es sagt**
Die Folge	Jede Verwendung braucht eine Prüfung, die niemand erzwingt

Der Sonderwert selbst ist nicht das Problem — `None` in Python leistet dasselbe. Das Problem ist, dass er im Typ nicht sichtbar ist. In Sprachen, die `T` und `T?` unterscheiden, prüft der Übersetzer; in C prüft niemand.

In C bleibt nur die Disziplin: Jede Funktion legt fest, ob sie Nullzeiger annimmt, und schreibt es auf. Der Rest ist Zusicherung — und im Betrieb hilft eine Zusicherung, die abgeschaltet wurde, nicht.

Aufgabe 3.1.

(a) Führen Sie eine Besitzregel ein.
(b) Verfolgen Sie einen Zugriff nach Freigabe.
(c) Bestimmen Sie, wann Zeiger überflüssig sind.

Lösung Schritt für Schritt.

(a) Eine Besitzregel einführen.
Die Vereinbarung für unser Modul.

Form	Besitz	Gültigkeit
`const T *`	**Aufrufer**	**nur im Aufruf**
`T *`	Aufrufer	nur im Aufruf
`T *` als Rückgabe	**Aufrufer**	**bis `t_frei`**
`..._uebernimm(T *)`	**Aufgerufener**	—

Vier Formen, vier Regeln — und sie stehen in einer Kopfzeile der Kopfdatei, nicht an jeder Funktion. Eine durchgehaltene Konvention ist schwächer als eine Sprachzusicherung und stärker als gar nichts.

Die Anwendung.

```
/* Aufrufer behaelt b; die Liste kopiert den Wert.  */
int liste_anfuegen(Liste *l, const Befund *b);

/* Aufrufer erhaelt Besitz; Freigabe mit verlauf_frei.
*/
Verlauf *verlauf_erzeuge(const Liste *l, uint16_t
code);
void verlauf_frei(Verlauf *v);

/* Die Liste uebernimmt den Puffer.  */
int liste_uebernimm_puffer(Liste *l, Befund *p, size_t
n);
```

Die Regel ist an der Signatur ablesbar, sobald man sie kennt — `const` heißt geliehen, ein zurückgegebener Zeiger heißt übergeben, ein Name mit `uebernimm` heißt abgegeben. Das ist keine Prüfung, aber es entfernt die Mehrdeutigkeit.

(b) Ein Zugriff nach Freigabe.
Der Fall. Ein Verlauf verweist auf Befunde der Liste — und die Liste wächst.

```
Verlauf *v = verlauf_erzeuge(&liste, KALIUM);
/* v->punkte zeigt in liste.daten hinein */
liste_anfuegen(&liste, &neuer);  /* realloc!  */
double x = v->punkte[0].wert;     /* Zugriff nach
Freigabe */
```

Verhalten	Wahrscheinlichkeit	
Funktioniert scheinbar	hoch	**alter Block noch lesbar**
Liefert alte Werte	hoch	stiller Fehler
Liefert fremde Daten	mittel	Block neu vergeben
Stürzt ab	gering	der beste Fall

Die letzte Zeile ist der beste Fall und der seltenste. Meist funktioniert es — monatelang, bis die Liste einmal an einer anderen Stelle umlagert. Dann liest der Verlauf Werte, die zu einem anderen Patienten gehören.

Die drei Auswege.

Weg	Speicher	Sicherheit	
Werte kopieren	×2	**sicher**	
Indizes statt Zeiger	gering	**sicher**	**empfohlen**
Liste einfrieren	gering	Vereinbarung	

Die mittlere Zeile ist die beste Antwort und wird selten gewählt: Ein Index bleibt gültig, wenn der Block umgelagert wird — ein Zeiger nicht. Dieselbe Einsicht wie in Band 20, Kapitel 6, dort für Positionen in Listen.

Der Preis: Ein Index ist ohne die zugehörige Liste bedeutungslos, und ein falscher Index wird nicht erkannt. Man tauscht einen lautlosen Fehler gegen einen anderen — aber gegen einen, den ein Bereichsprüfer findet.

(c) Wann Zeiger überflüssig sind.

Zweck	Zeiger nötig?	Alternative
Große Struktur übergeben	ja	
Ändern beim Aufrufer	ja	
Kleiner Wert übergeben	**nein**	**als Wert**
Element in einem Feld	**nein**	**Index**
Optionaler Rückgabewert	oft	**Verbund mit Kennzeichen**
Dynamische Struktur	ja	

Die dritte Zeile widerspricht einer verbreiteten Gewohnheit: Ein `int` oder ein Verbund von sechzehn Byte wird als Wert übergeben, nicht als Zeiger — das ist schneller und schließt drei Fehlerklassen aus. Die Grenze liegt bei etwa der doppelten Registergröße.

Der Rückgabeverbund.

```
typedef struct { bool gefunden; Befund wert; }
BefundOpt;

BefundOpt letzter_vor(const Liste *l, time_t t);
```

	Zeiger oder `NULL`	Verbund
Prüfung vergessbar	**ja**	auch, aber sichtbar
Besitzfrage	offen	**keine — Wert**
Lebensdauer	unklar	beim Aufrufer
Kosten	8 B	40 B kopiert

Vierzig Byte Kopie gegen drei beseitigte Unklarheiten. Bei einem Aufruf je Sekunde ist die Wahl offensichtlich; bei einer Million Aufrufen je Sekunde nicht. Auch das ist eine Größenordnungsfrage.

Der Zeiger ist nicht das Problem

Ursache	Trägt?	
Zeiger als Begriff	nein	**jede Sprache hat sie**
Fehlende Längenangabe	**ja**	Kapitel 4
Fehlende Lebens-dauerangabe	**ja**	
Keine Prüfung	ja	

Auch Python hat Verweise — es prüft nur ihre Gültigkeit und zählt die Verwendungen. Der Unterschied ist nicht der Begriff, sondern dass in C die drei Zusatzangaben fehlen, die einen Verweis benutzbar machen: wie viele, wie lange, wem gehört er.

Deshalb lautet die praktische Regel dieses Kapitels: Reiche Zeiger nie allein herum. Ein Zeiger gehört mit seiner Länge in einen Verbund, und seine Lebensdauer gehört in die Vereinbarung — Kapitel 4 führt das vor.

Hoare[11] nannte den Nullzeiger im Rückblick seinen teuersten Fehler; Hanson[10] zeigt, wie man Besitz in C-Schnittstellen sauber festlegt; van der Linden[26] erklärt die Zeigerfallen mit den Beispielen, an denen sie tatsächlich auftreten.

Part II

Der erste Entwurf in C

4 Zeichenketten

Die teuerste Entscheidung der Sprache

	C	Zeiger und Länge
Länge ermitteln	$\mathcal{O}(n)$ **suchen**	$\mathcal{O}(1)$ ablesen
Teilkette	kopieren nötig	**Ausschnitt**
Nullbyte im Inhalt	**unmöglich**	erlaubt
Überlauf	leicht	prüfbar
Kosten	1 Byte	8–16 Byte

Die Entscheidung, das Ende durch ein Nullbyte zu markieren, sparte 1972 ein bis zwei Byte je Zeichenkette — bei Rechnern mit wenigen Kilobyte eine ernsthafte Erwägung. Sie hat seither mehr Sicherheitslücken verursacht als jede andere Einzelentscheidung der Informatik.

Was daraus folgt.

Funktion	Gefahr	
`strcpy`	**keine Grenze**	**nie verwenden**
`strcat`	keine Grenze	nie verwenden
`sprintf`	keine Grenze	nie verwenden
`strncpy`	**schließt evtl. nicht ab**	Falle
`snprintf`	begrenzt	Rückgabe prüfen

Die vierte Zeile ist die tückischste: `strncpy` *sieht nach der sicheren Fassung aus und schreibt bei vollem Puffer* kein *abschließendes Nullbyte. Wer sie benutzt, ohne das zu wissen, hat den Fehler nur verschoben.*

Was `snprintf` zurückgibt

Nicht	Die Zahl der geschriebenen Zeichen
Sondern	**Die Zahl, die geschrieben worden *wäre***
Die Prüfung	`if (n >= groesse) /* abgeschnitten */`

Das ist genau richtig entworfen — man erfährt, ob abgeschnitten wurde, und wie viel Platz nötig gewesen wäre. Und es ist die am häufigsten ignorierte Rückgabe der Standardbibliothek: Ein abgeschnittener Patientenname wird zu einem falschen Patientennamen.

Der zweite Entwurf: Ausschnitte

```
typedef struct { const char *zeichen; size_t laenge; }
Str;

#define STR(s) ((Str){ (s), sizeof(s) - 1 })

static inline bool str_gleich(Str a, Str b) {
return a.laenge == b.laenge &&
memcmp(a.zeichen, b.zeichen, a.laenge) == 0;
}

static inline Str str_teil(Str s, size_t ab, size_t
n) {
if (ab > s.laenge) return (Str){ s.zeichen, 0 };
if (n > s.laenge - ab) n = s.laenge - ab;
return (Str){ s.zeichen + ab, n };
}
```

Vierzehn Zeilen, und die Hälfte der Zeichenkettenfehler ist ausgeschlossen. Der Ausschnitt kopiert nicht — er verweist mit Länge. Das ist dieselbe Bauform wie in modernen Sprachen, und sie ist in C seit vierzig Jahren möglich.

Die Bilanz.

Vorgang	`char *`	`Str`
Länge	$\mathcal{O}(n)$	$\mathcal{O}(\mathbf{1})$
Teilkette	**kopieren**	$\mathcal{O}(1)$**, ohne Kopie**
Vergleich	sicher	sicher
Überlauf möglich	**ja**	**nein**
Mit Bibliothek nutzbar	ja	Umwandlung nötig

Die letzte Zeile ist der Preis: Die Standardbibliothek erwartet nullterminierte Ketten. An der Grenze zu ihr muss umgewandelt werden — und genau dort, an einer Stelle, prüft man einmal statt überall.

Der Lebensdauerfehler beim Ausschnitt

```
Str hole_namen(void) {
char puffer[64];
lies_in(puffer, sizeof puffer);
return (Str){ puffer, strlen(puffer) };  /* FALSCH
*/
}
```

Was geschieht	Sichtbar?	
Keller wird freigegeben	nein	
Zeiger bleibt	nein	
Funktioniert zunächst	**ja**	**Werte noch da**
Bricht beim nächsten Aufruf	später	anderswo

Ein Ausschnitt verschiebt das Kopieren — er beseitigt nicht die Frage nach der Lebensdauer, sondern macht sie wichtiger. Der Verweis darf sein Ziel nicht überleben, und nichts erzwingt das. Übersetzer warnen bei diesem Fall inzwischen; bei einem Umweg über eine zweite Funktion nicht mehr.

Aufgabe 4.1.

(a) Bauen Sie einen sicheren Zusammenbau.
(b) Rechnen Sie die Kosten des Kopierens aus.
(c) Untersuchen Sie die Zerlegung einer Befundzeile.

Lösung Schritt für Schritt.

(a) Sicherer Zusammenbau.
Der erste Entwurf.

```
char zeile[128];
strcpy(zeile, name);
strcat(zeile, ":  ");
strcat(zeile, wert);            /* Ueberlauf moeglich
*/
```

Der zweite Entwurf: ein Aufbaupuffer.

```
typedef struct { char *ziel; size_t frei; bool voll; }
Bau;

static void bau_anfuege(Bau *b, Str s) {
if (b->voll) return;
if (s.laenge >= b->frei) { b->voll = true; return; }
memcpy(b->ziel, s.zeichen, s.laenge);
b->ziel += s.laenge;
b->frei -= s.laenge;
}
```

Der Kunstgriff steht in Zeile vier: Sobald einmal kein Platz war, tut jeder weitere Anhang nichts. Man muss also nicht nach jedem Schritt prüfen, sondern einmal am Ende — und vergisst die Prüfung damit seltener.

	Erster	**Zweiter**	
Prüfstellen	0	**1**	am Ende
Überlauf möglich	**ja**	**nein**	
Abschneiden erkennbar	nein	**ja**	`voll`
Zeilen	4	13	

Die dritte Zeile ist der klinisch wichtige Punkt: Ein stillschweigend abgeschnittener Name ist ein falscher Name. Das Kennzeichen `voll` macht aus einem lautlosen Fehler einen erkennbaren.

(b) Kosten des Kopierens.
Der Fall. Eine Befunddatei zerlegen, 4 Millionen Zeilen.

Vorgehen	Belegungen	Dauer	
Je Feld kopieren	**$2{,}8 \cdot 10^7$**	**4,1 s**	
Zeile kopieren	$4 \cdot 10^6$	1,2 s	
Ausschnitte	**0**	**0,31 s**	13×

> *Faktor dreizehn allein durch das Vermeiden von Kopien. Die Ausschnitte zeigen direkt in den Lesepuffer — solange dieser gültig ist. Damit ist die Lebensdauerfrage aus dem Warnkasten wieder da, hier aber beherrschbar: Die Verarbeitung endet vor dem nächsten Lesen.*

Das ist die wiederkehrende Abwägung dieses Bandes: Wer nicht kopiert, ist schneller und übernimmt die Verantwortung für die Lebensdauer. Wer kopiert, ist langsamer und sicher.

(c) Eine Befundzeile zerlegen.

```
/* "12345|KAL|4.2|mmol/l|2026-03-14T08:30" */
static bool zerlege(Str zeile, Str *felder, size_t n) {
size_t k = 0, ab = 0;
for (size_t i = 0; i <= zeile.laenge; i++) {
if (i == zeile.laenge || zeile.zeichen[i] == '|') {
if (k == n) return false;
felder[k++] = str_teil(zeile, ab, i - ab);
ab = i + 1;
}
}
return k == n;
}
```

Die Grenzfälle.

Eingabe	Erwartet	Verhalten
Leere Zeile	1 leeres Feld	richtig
Zu wenige Felder	`false`	richtig
Zu viele Felder	`false`	richtig
Leeres Feld mittendrin	**leerer Ausschnitt**	**richtig**
Trenner am Ende	$n{+}1$ Felder	erkannt

Fünf Grenzfälle, alle richtig behandelt — und zwar weil die Schleifenbedingung `i <= zeile.laenge` *lautet statt* `<`*. Ein Zeichen Unterschied entscheidet, ob das letzte Feld erkannt wird. Genau solche Stellen findet man mit Randfallprüfungen, nicht durch Lesen.*

Bemerkenswert ist, was nicht *nötig war: keine Belegung, keine Kopie, keine Freigabe, kein* `strtok` *mit seinem verborgenen Zustand. Das ist der Gewinn der Ausschnittsdarstellung.*

Kodierung ist eine eigene Ebene

Frage	`char` beantwortet	
Wie viele Bytes?	ja	
Wie viele Zeichen?	**nein**	**UTF-8:** 1–4 **Bytes**
Wie breit dargestellt?	nein	noch etwas anderes
Gleich bei Vergleich?	**nein**	Normalformen

Ein Umlaut in einem Patientennamen ist in UTF-8 zwei Bytes. Wer auf 60 Bytes abschneidet, kann mitten in ein Zeichen schneiden — und erzeugt eine Kette, die kein gültiger Text mehr ist. Die vierte Zeile ist noch schlimmer: Derselbe Name kann in zwei verschiedenen Bytefolgen vorliegen und vergleicht sich als ungleich.

C hilft dabei nicht. Wer mit Namen arbeitet, braucht eine Bibliothek für Kodierung und Normalisierung — und die Erkenntnis, dass „Zeichenkette“ drei verschiedene Dinge bedeutet, ist hier so wichtig wie die Längenfrage.

Koenig[17] sammelte die Zeichenkettenfallen, bevor sie zu Sicherheitslücken wurden; Seacord[23] behandelt sie aus heutiger Sicht mit den zugehörigen Angriffen; Kernighan und Ritchie[14] beschreiben die ursprüngliche Entwurfsentscheidung und ihre Gründe.

5 Fehler ohne Ausnahmen

Vier Wege, einen Fehler zu melden

Weg	Vorteil	Nachteil
Rückgabewert	**sichtbar**	**ignorierbar**
Nutzlast über Zeiger	Rückgabe frei für Code	mehr Argumente
Globale Variable	—	**nicht fadensicher**
Sonderwert im Wertebereich	knapp	**mehrdeutig**

Die letzte Zeile ist die verbreitetste und schlechteste: Wenn eine Funktion -1 für „Fehler“ zurückgibt und -1 zugleich ein gültiger Messwert ist, kann der Aufrufer beides nicht unterscheiden. In der Klinik heißt das: Ein Fehler wird als Wert gebucht.

Die verbreitete Empfehlung und ihre Grenze.

Die Regel	Rückgabewert ist der Fehlercode, Ergebnisse gehen über Zeiger
Der Gewinn	**Eindeutig, keine Mehrdeutigkeit**
Der Preis	Der Aufrufer muss eine Variable bereitstellen, bevor er weiß, ob sie gefüllt wird

Die dritte Zeile ist unschön und beherrschbar. Die Alternative — ein Verbund aus Kennzeichen und Wert — ist sauberer und kostet eine Kopie. Bei kleinen Ergebnissen ist sie vorzuziehen, bei großen nicht.

Das eigentliche Problem: Aufräumen

```
int lade(const char *pfad, Liste *l) {
FILE *f = fopen(pfad, "r");
if (!f) return -EIO;
char *puffer = malloc(GROESSE);
if (!puffer) { fclose(f); return -ENOMEM; }
Index *ix = index_erzeuge();
if (!ix) { free(puffer); fclose(f); return -ENOMEM; }
/* ...  */
}
```

Bei drei Betriebsmitteln braucht man drei verschiedene Aufräumfolgen. Bei sechs sind es sechs — und die fünfte wird falsch geschrieben. Das ist die häufigste Quelle für Speicherlecks in C, und sie ist rein struktureller Natur.

Der zweite Entwurf: ein einziger Ausgang.

```
int lade(const char *pfad, Liste *l) {
int rc = -ENOMEM;
FILE *f = NULL; char *puffer = NULL; Index *ix = NULL;

f = fopen(pfad, "r");
if (!f) { rc = -EIO; goto raus; }
puffer = malloc(GROESSE);
if (!puffer) goto raus;
ix = index_erzeuge();
if (!ix) goto raus;
/* ...  */
rc = 0;

raus:
index_frei(ix);
free(puffer);
if (f) fclose(f);
return rc;
}
```

Ein Aufräumblock statt sechs. Voraussetzung sind die Anfangswerte `NULL` und Freigabefunktionen, die `NULL` vertragen — `free(NULL)` ist erlaubt, eigene Funktionen sollten es genauso halten. Dann ist der Block immer derselbe, gleichgültig wo man aussteigt.

Der Sprung, der erlaubt ist

Verwendung	Vertretbar?	
Aufräumen am Funktionsende	**ja**	**Standard im Kernbereich**
Aus geschachtelten Schleifen	ja	
Rückwärts springen	**nein**	
Zwischen Zweigen	nein	unlesbar

Dijkstras Kritik von 1968 galt beliebigen Sprüngen, die den Programmablauf unverfolgbar machen. Ein Vorwärtssprung an genau eine Marke am Funktionsende ist kein solcher Fall — er bildet nach, was andere Sprachen mit `finally` anbieten.

Aufgabe 5.1.

(a) Zählen Sie die Aufräumpfade.

(b) Entwerfen Sie einen Ergebnistyp.

(c) Untersuchen Sie ignorierte Rückgabewerte.

Lösung Schritt für Schritt.

(a) Aufräumpfade zählen.

Betriebsmittel	Verzweigt	Freigabe-zeilen	Mit `goto`
1	1	1	1
3	3	**6**	**3**
5	5	**15**	5
8	8	**36**	**8**

Die Zahl der Freigabezeilen wächst quadratisch — $n(n+1)/2$ — weil beim k-ten Fehlschlag $k-1$ Betriebsmittel freizugeben sind. Mit einem Ausgang wächst sie linear. Bei acht Betriebsmitteln sind das sechsunddreißig gegen acht Zeilen, und jede der sechsunddreißig kann falsch sein.

Die Fehlerrate.

Betriebsmittel	**Stellen**	***P*(Fehler)**	
3	6	$\approx 6\,\%$	bei $f = 0{,}01$
5	15	$\approx 14\,\%$	
8	36	$\approx$ **30** %	fast sicher

Bei acht Betriebsmitteln und einer Fehlerrate von einem Prozent je Stelle enthält fast jede dritte solche Funktion ein Leck. Wieder die Rechnung aus Band 19: Die Zahl der Gelegenheiten schlägt die Sorgfalt.

(b) Ein Ergebnistyp.
Die Aufgabe. Der Ergebnistyp aus Band 20, Kapitel 2 — Zahlwert, Grenzwert, qualitativ, nicht auswertbar.

```
typedef enum { ERG_ZAHL, ERG_GRENZE, ERG_QUAL, ERG_NA }
ErgArt;

typedef struct {
ErgArt art;
union {
struct { int64_t betrag; int8_t exp; } zahl;
struct { bool kleiner; int64_t schranke; } grenze;
Str qual;
Str grund;
} u;
} Ergebnis;
```

Ein Verbund mit Kennzeichen — was in Band 20 vier Klassen waren. Zwei Entscheidungen stecken darin: Das Kennzeichen liegt im *Verbund, damit es nicht verlorengehen kann. Und die Zahl ist eine Ganzzahl mit Zehnerexponent, kein* `double` *— Kapitel 8 begründet das.*

Die Fallunterscheidung.

```
switch (e.art) {
case ERG_ZAHL:   return ...;
case ERG_GRENZE: return ...;
case ERG_QUAL:   return ...;
case ERG_NA:     return ...;
}  /* kein default:  der Uebersetzer meldet fehlende
Faelle */
```

	Mit `default`	Ohne
Neue Art hinzu	**still ignoriert**	**Warnung**
Vergessener Fall	nie bemerkt	beim Übersetzen

Das Weglassen von `default` ist die einzige Möglichkeit, in C eine Vollständigkeitsprüfung zu bekommen — vorausgesetzt, man übersetzt mit den passenden Warnungen und behandelt sie als Fehler. Kapitel 9 führt das aus.

(c) Ignorierte Rückgabewerte.

Funktion	Ignoriert wird	Folge
`malloc`	`NULL`	Absturz
`fclose`	**Schreibfehler**	**Daten verloren**
`snprintf`	Abschneidung	falscher Text
`fwrite`	Kurzschreiben	**Datei unvollständig**
`scanf`	Zahl der Felder	uninitialisiert

Die zweite Zeile überrascht viele: `fclose` kann fehlschlagen, weil erst dort die Puffer geschrieben werden. Wer den Rückgabewert ignoriert, erfährt nie, dass die Datei nicht vollständig auf der Platte steht — bei Befunddaten ein ernster Fall.

Die Abhilfe.

```
__attribute__((warn_unused_result))
int liste_anfuegen(Liste *l, const Befund *b);

/* oder in C23:  */
[[nodiscard]] int liste_anfuegen(Liste *l, const Befund
*b);
```

Maßnahme	Aufwand	Wirkung
Durchsicht	hoch	unvollständig
Regel im Handbuch	gering	keine
`nodiscard`	**gering**	**vollständig**

Eine Markierung an der Signatur, und der Übersetzer meldet jede ignorierte Rückgabe. Das ist der Unterschied zwischen einer Konvention und einer Prüfung — und er kostet fünf Minuten je Kopfdatei.

Fehlerbehandlung ist der halbe Code

Modul	Zeilen gesamt	davon Fehler	Anteil
Einlesen	340	**156**	**46** %
Fachlogik	620	84	14 %
Ausgabe	280	118	42 %

An den Rändern besteht fast die Hälfte des Codes aus Fehlerbehandlung, in der Fachlogik ein Siebtel. Das ist ein weiteres Argument für die Trennung aus Band 20, Kapitel 6: Wo die Fachlogik keine Betriebsmittel anfasst, hat sie auch keine Fehlerpfade.

Und es erklärt, warum C-Code länger wirkt, als er ist: Ein erheblicher Teil beantwortet Fragen, die in anderen Sprachen die Laufzeitumgebung beantwortet — nicht schlechter, nur sichtbar.

Gustedt[9] behandelt die heutigen Sprachmittel für Fehlerbehandlung; Kernighan und Pike[13] zeigen den Umgang mit Fehlern als Handwerksregel; Hanson[10] entwirft Schnittstellen, in denen Fehlerfälle Teil des Vertrags sind.

6 Datenstrukturen von Hand

Was man selbst bauen muss

Struktur	In Python	In C
Wachsendes Feld	`list`	**selbst**
Abbildung	`dict`	**selbst**
Menge	`set`	selbst
Warteschlange	`deque`	selbst
Sortierte Folge	`sorted`	`qsort`

Die zweite Zeile ist die folgenreichste. Ein Wörterbuch ist in Python eine eingebaute Selbstverständlichkeit und in C ein Nachmittag Arbeit — mit Streufunktion, Kollisionsbehandlung, Wachstum und Löschmarken. Deshalb wird in C oft linear gesucht, wo eine Abbildung richtig wäre.

Wann lineares Suchen genügt.

n	Linear	Streutabelle	Besser
8	**4** ns	18 ns	linear
50	**22** ns	19 ns	gleich
500	210 ns	**20** ns	Tabelle
10^5	42 µs	**24** ns	**Tabelle**

Bis etwa fünfzig Einträgen ist lineares Suchen in einem zusammenhängenden Feld nicht langsamer — weil die Zwischenspeicherzeilen alles auf einmal liefern, während die Streutabelle einen unvorhersagbaren Sprung macht. Das ist die Konstante aus Kapitel 2.

Die generische Frage

C kennt keine Typparameter. Es gibt drei Auswege, und alle drei kosten etwas.

Weg	Typprüfung	Geschwindigkeit	Lesbar
`void *`	**keine**	Umwege	gut
Makros	teilweise	**voll**	**schlecht**
Für jeden Typ neu	**voll**	voll	gut

Die dritte Zeile ist die unpopuläre und meist richtige: Zwei Listen von Hand für die zwei Typen, die man tatsächlich braucht, statt einer generischen für alle denkbaren. Das ist die Regel der drei Fälle aus Band 20, Kapitel 14, hier besonders scharf — denn generischer Code kostet in C mehr als anderswo.

Was `void *` kostet.

Posten	Wirkung	
Element als Zeiger	**eigene Belegung**	**Kap. 2**
Nicht zusammenhängend	Zwischenspeicher	≈ 40×
Kein Typvergleich	**Fehler zur Laufzeit**	
Vergleichsfunktion	Aufruf je Vergleich	nicht einbettbar

Eine generische Liste in C hat dieselbe Speicheranordnung wie eine Python-Liste — Zeiger auf verstreute Objekte — und verliert damit genau den Vorteil, für den man C gewählt hat.

Makros als dritter Weg

```
#define FELD(T) struct { T *daten; size_t n, kap;
}

#define ANFUEGE(f, wert) ( \
((f).n == (f).kap ?  feld_wachse(&(f).daten,
&(f).kap, \
sizeof *(f).daten) :  0), \
(f).daten[(f).n++] = (wert))
```

Volle Geschwindigkeit, echte Typen, zusammenhängender Speicher — und Fehlermeldungen, die auf die Makrozeile zeigen statt auf die Verwendung. Bei einem Tippfehler bekommt man eine Seite unverständlicher Diagnostik.

Die Abwägung: Makros lohnen für wenige, viel benutzte Strukturen. Für alles andere schreibt man die zwei oder drei Ausprägungen von Hand und spart sich die Diagnostik.

Aufgabe 6.1.

(a) Bauen Sie eine Streutabelle für Messgrößen.
(b) Vergleichen Sie Anordnungen für den Verlauf.
(c) Bestimmen Sie, wann eine Struktur zu viel ist.

Lösung Schritt für Schritt.

(a) Eine Streutabelle.
Die Aufgabe. Code → Messgröße, etwa 2000 Einträge, nur Nachschlagen.

```
typedef struct { uint16_t code; Messgroesse g; }
Eintrag;

typedef struct {
Eintrag *tab;    /* Zweierpotenz */
size_t maske;    /* groesse - 1 */
size_t belegt;
} Tabelle;

static const Messgroesse *finde(const Tabelle *t,
uint16_t c) {
size_t i = streue(c) & t->maske;
while (t->tab[i].code != 0) {
if (t->tab[i].code == c) return &t->tab[i].g;
i = (i + 1) & t->maske;
}
return NULL;
}
```

Offene Adressierung mit linearer Sondierung: Bei Kollision wird die nächste Zelle genommen — die liegt in derselben Zwischenspeicherzeile. Verkettung mit Listen wäre lehrbuchmäßig und in der Praxis um Faktor drei langsamer.

Die drei Entscheidungen darin.

Entscheidung	Gewählt	Warum
Größe	**Zweierpotenz**	**Maske statt Modulo**
Kollision	lineare Sondierung	Zwischenspeicher
Leerwert	`code == 0`	0 **als Code verboten**
Füllgrad	max. 70 %	darüber Häufung

Die dritte Zeile ist eine Annahme über die Daten, die aufgeschrieben gehört: Wenn je ein Code 0 auftritt, findet die Tabelle ihn nie. Solche stillen Voraussetzungen sind in C häufiger als anderswo — und gehören in eine Zusicherung.

Füllgrad	Sondierungen	Dauer	
50 %	1,5	19 ns	
70 %	2,5	**22** ns	Grenze
85 %	4,0	31 ns	
95 %	**10,5**	**78** ns	Häufung

Ab siebzig Prozent Füllgrad steigt die Zahl der Sondierungen überproportional. Das ist der Grund für die Faustregel — und dafür, dass man rechtzeitig verdoppelt statt zu warten, bis es eng wird.

(b) Anordnungen für den Verlauf.
Die Aufgabe. Aus 10^6 Befunden die einer Größe, zeitlich sortiert.

Anordnung	Suchen	Anfügen	Speicher
Feld, unsortiert	**4,1** ms	**1** µs	32 MB
Feld, nach Zeit sortiert	4,1 ms	260 µs	32 MB
Je Größe ein Feld	**9** µs	1 µs	**34** MB
Verkettete Liste	83 ms	1 µs	56 MB

Die dritte Zeile gewinnt in allen Spalten außer zwei Megabyte Speicher — Faktor 450 *beim Suchen. Der Grund ist keine bessere Datenstruktur, sondern eine Vorsortierung nach dem Zugriffsmuster: Man fragt fast immer nach einer Größe.*

Wieder Wirths Rangordnung: Die Umstellung war keine Verfahrensverbesserung, sondern eine Änderung der Anordnung. Das Suchverfahren blieb dasselbe — es durchläuft nur noch ein Tausendstel der Daten.

(c) Wann eine Struktur zu viel ist.

Lage	Eigene Struktur?	
$n < 50$, selten	**nein**	**lineares Feld**
$n < 50$, sehr häufig	nein	Feld ist schneller
n groß, nur lesen	sortiertes Feld	**Binärsuche**
n groß, ändern	ja	Streutabelle
Reihenfolge zählt	ja	

Die dritte Zeile wird übersehen: Für eine Nachschlagetabelle, die sich nie ändert, ist ein sortiertes Feld mit Binärsuche einfacher als eine Streutabelle, braucht keinen Füllgrad, keine Kollisionsbehandlung und keine Löschmarken — bei zwanzig Zeilen Code.

	Zeilen	Suchen	Fehler-quellen
Sortiertes Feld	**20**	34 ns	**2**
Streutabelle	130	**22** ns	**9**

Zwölf Nanosekunden Gewinn gegen hundertzehn Zeilen und sieben zusätzliche Fehlerquellen. Wenn die Tabelle einmal beim Start gefüllt wird und danach nur gelesen, ist die Wahl eindeutig — und fällt trotzdem meist zugunsten der Streutabelle aus, weil sie richtiger klingt.

Selbstgebaute Strukturen sind Bestand

Posten	**Einmalig?**	
Schreiben	ja	≈ 4 h
Prüfen	ja	≈ 6 **h — Randfälle**
Pflegen	**nein**	**jahrelang**
Nachfolger einarbeiten	nein	jedes Mal

Eine selbstgebaute Streutabelle ist nicht in vier Stunden fertig, sondern in vier Stunden lauffähig. Danach ist sie zehn Jahre lang Code, den jeder Nachfolger verstehen muss — und der bei jeder Erweiterung neu geprüft werden will.

Deshalb ist die Frage nie nur „kann ich das bauen“, sondern „brauche ich es“. Eine geprüfte Bibliothek einzubinden ist auch in C oft die richtige Antwort — und die Zurückhaltung dagegen ist Gewohnheit, nicht Argument.

Wirth[28] zeigt die Grundstrukturen und ihre Kompromisse; Bentley[2] führt vor, wie die Wahl der Anordnung ein Programm um Größenordnungen ändert; Hanson[10] liefert saubere C-Umsetzungen mit durchdachten Schnittstellen.

Part III

Die Fallen

7 Undefiniertes Verhalten

Was undefiniert wirklich bedeutet

Nicht	„Es passiert irgendetwas Zufälliges“
Nicht	„Es hängt vom Prozessor ab“
Sondern	**Der Standard stellt keine Anforderung — an das *ganze* Programm**

Der Unterschied ist entscheidend: Ein Programm mit undefiniertem Verhalten ist nicht „an dieser Stelle unvorhersagbar“. Es hat überhaupt keine festgelegte Bedeutung mehr — auch nicht für die Zeilen davor. Der Übersetzer darf daraus beliebige Schlüsse ziehen.

Die drei Arten, die verwechselt werden.

Art	Bedeutet	Beispiel
Undefiniert	**keine Anforderung**	**Überlauf bei `int`**
Unspezifiziert	mehrere erlaubte Verhalten	**Auswertungs-reihenfolge**
Umsetzungs-abhängig	festgelegt und dokumentiert	Größe von `int`

Nur die erste Zeile ist gefährlich. Die dritte ist harmlos, solange man nachschlägt; die zweite erzeugt Überraschungen, aber das Programm bleibt sinnvoll. Die erste hebt die Bedeutung des Programms auf.

Wie der Übersetzer daraus schließt

```
int mittel(int a, int b) {
int s = a + b;          /* kann ueberlaufen */
if (s < 0) fehler();    /* Pruefung */
return s / 2;
}
```

1	Überlauf bei vorzeichenbehafteten Ganzzahlen ist undefiniert.
2	Also darf der Übersetzer annehmen, dass er nicht vorkommt.
3	Wenn $a, b \geq 0$ und kein Überlauf, dann ist $s \geq 0$.
4	**Also ist `s < 0` immer falsch.**
5	**Also wird die Prüfung entfernt.**

Der Übersetzer hat nichts falsch gemacht — jeder Schritt folgt aus dem Standard. Die Sicherheitsprüfung verschwindet aus dem übersetzten Programm, und zwar lautlos. Genau so sind reale Sicherheitslücken entstanden.

Die häufigsten Fälle

Fall	Bemerkt?	Findet
Überlauf, vorzeichenbehaftet	**nie**	**UBSan**
Zugriff über Feldgrenze	selten	ASan
Zugriff nach Freigabe	selten	ASan
Uninitialisierte Variable	**selten**	MSan
Nullzeigerzugriff	meist	
Verschiebung um $\geq$ Breite	nie	UBSan
Typverletzung beim Alias	**nie**	**schwer**

Die rechte Spalte ist die Botschaft dieses Kapitels: Sechs von sieben Fällen findet ein Werkzeug in Sekunden. Keiner davon wird durch Sorgfalt zuverlässig vermieden — und keiner durch Lesen gefunden.

Die Aliasregel

```
float f = 1.0f;
uint32_t bits = *(uint32_t *)&f;  /* undefiniert
*/

uint32_t bits;                    /* richtig:  */
memcpy(&bits, &f, sizeof bits);
```

Vorgehen	Erlaubt?	
Zeiger umdeuten	**nein**	**Aliasregel**
Über `union`	in C ja	in C++ nein
`memcpy`	**ja**	**wird wegoptimiert**
Über `char *`	ja	Ausnahme im Standard

Der `memcpy`-Weg sieht aus, als koste er eine Kopie, und erzeugt bei jedem heutigen Übersetzer denselben Maschinenbefehl wie die verbotene Umdeutung. Man bekommt Richtigkeit umsonst — und muss es nur wissen.

Aufgabe 7.1.

(a) Finden Sie undefiniertes Verhalten in einer Prüfung.
(b) Rechnen Sie die Kosten der Werkzeuge aus.
(c) Bestimmen Sie, wie man überlaufsicher rechnet.

Lösung Schritt für Schritt.

(a) Undefiniertes Verhalten in einer Prüfung.
Der Code aus dem Messdatenerfasser.

```
int pruefe(int *werte, int n, int schwelle) {
int summe = 0;
for (int i = 0; i <= n; i++)     /* Fehler 1 */
summe += werte[i];          /* Fehler 2 */
if (summe + schwelle < 0)       /* Fehler 3 */
return -1;
return summe / n;              /* Fehler 4 */
}
```

Nr.	Fehler	Art	Findet
1	<= statt <	**Grenzüberschreitung**	ASan
2	Zugriff auf `werte[n]`	undefiniert	ASan
3	**Überlauf der Summe**	**undefiniert**	**UBSan**
4	**Division durch** 0 **bei** $n = 0$	undefiniert	UBSan

Vier Fehler in sieben Zeilen — und die Funktion läuft bei den üblichen Eingaben tadellos. Fehler 3 ist der tückischste: Die Prüfung soll einen Überlauf abfangen und ist *selbst der Überlauf. Sie wird vom Übersetzer entfernt.*

Der zweite Entwurf.

```
int pruefe(const int32_t *werte, size_t n, int32_t
schwelle,
int32_t *ergebnis) {
if (n == 0) return -EINVAL;
int64_t summe = 0;          /* breiter:  kein Ueberlauf
*/
for (size_t i = 0; i < n; i++)
summe += werte[i];
if (summe + schwelle < 0) return -ERANGE;
*ergebnis = (int32_t)(summe / (int64_t)n);
return 0;
}
```

Vier Änderungen: `size_t` *für die Länge, Prüfung auf null, breiterer Summentyp, Ergebnis über Zeiger. Der Summentyp ist der Kunstgriff — bei* 2^{31} *Werten zu je* 2^{31} *kann eine* 64*-bit-Summe nicht überlaufen.*

(b) Kosten der Werkzeuge.

Werkzeug	Laufzeit	Speicher	Findet
ASan	×2	×3	**Speicherfehler**
UBSan	×1,2	×1	**undefiniertes Verh.**
MSan	×3	×2	Uninitialisiertes
TSan	×10	×7	Wettläufe
Valgrind	×30	×4	Speicher, ohne Neuübersetzung

Die zweite Zeile ist bemerkenswert günstig: Zwanzig Prozent Laufzeit für die Prüfung auf undefiniertes Verhalten. In manchen Häusern läuft UBSan deshalb sogar im Betrieb — mit Protokollierung statt Abbruch.

Der Ertrag am Fall.

Vorgehen	Aufwand	Gefundene Fehler	
Durchsicht	4 h	1 von 4	
Prüfungen ohne Werkzeug	2 h	1 von 4	
Prüfungen mit Sanitizern	**2,5 h**	**4 von 4**	**vollständig**

Eine halbe Stunde mehr Aufwand für die vierfache Ausbeute. Die Werkzeuge finden genau die Fehlerklassen, die dem Auge entgehen — und sie tun es bei jedem Prüflauf wieder, ohne dass jemand daran denken muss.

(c) Überlaufsicher rechnen.

Weg	Sicher?	
Vorher prüfen	**ja**	`a > MAX - b`
Breiterer Typ	ja	**wenn verfügbar**
Vorzeichenlos	definiert	Umlauf, oft falsch
Eingebaute Prüffunktionen	**ja**	`__builtin_add_overflow`
Nachher prüfen	**nein**	**zu spät**

Die dritte Zeile ist die häufigste Fehlannahme: Vorzeichenloser Überlauf ist definiert *— er läuft um — und deshalb kein undefiniertes Verhalten. Er ist trotzdem meist ein Fehler, weil aus einer Subtraktion eine riesige Zahl wird, die dann als Länge dient.*

```
size_t n = anzahl - 1;        /* bei anzahl == 0:  */
for (size_t i = 0; i < n; i++) /* 18 Trillionen
Durchlaeufe */
```

Kein undefiniertes Verhalten, kein Werkzeugbefund — und trotzdem ein schwerer Fehler. Die Regel lautet deshalb: Bei vorzeichenlosen Typen ist jede Subtraktion eine Gefahrenstelle, und man prüft vorher statt nachher.

Der Optimierer wird stärker

Zeitraum	Ausnutzung	
1990er	gering	**Code lief „wie erwartet“**
2000er	mittel	erste Überraschungen
2010er	hoch	**Prüfungen verschwinden**
Heute	sehr hoch	übergreifend

Alter Code, der jahrzehntelang lief, bricht bei einem neuen Übersetzer — nicht weil der Übersetzer schlechter geworden wäre, sondern weil er die Freiheiten nutzt, die der Standard ihm immer gab. Das trifft besonders langlebige Systeme, und ein KIS ist ein solches.

Daraus folgt: Beim Wechsel des Übersetzers gehören die Sanitizer als Erstes gestartet. Was vorher lief und jetzt nicht mehr, war meist schon vorher falsch — man hat es nur nicht bemerkt.

Lattner[18] erklärt in einer dreiteiligen Darstellung, wie Übersetzer aus undefiniertem Verhalten schließen; Regehr[21] sammelt die Fälle mit Beispielen aus echten Systemen; der Standard[12] selbst führt die über zweihundert Fälle auf.

8 Ganzzahlen und Gleitkomma

Die stille Umwandlung

```
int a = -1;
unsigned b = 1;
if (a < b) ...          /* falsch:  a wird zu
4294967295 */
```

Die Regel	Bei gemischten Typen wird zum „größeren“ umgewandelt
Der Haken	**Vorzeichenlos schlägt vorzeichenbehaftet bei gleicher Breite**
Die Folge	Ein negativer Wert wird zu einer sehr großen Zahl

Kein undefiniertes Verhalten, keine Warnung ohne besondere Einstellung, keine Werkzeugmeldung — und ein Vergleich, der das Gegenteil dessen liefert, was dasteht. Das ist die tückischste Falle dieses Kapitels, weil sie vollständig regelkonform ist.

Wo sie zuschlägt.

Muster	Problem	
`i < strlen(s)`	`size_t`	meist harmlos
`n - 1` bei `n == 0`	**Umlauf**	**riesige Zahl**
`if (laenge < 0)`	nie wahr	**Prüfung wirkungslos**
Rückgabe -1 in `size_t`	**größter Wert**	

Die dritte Zeile ist die gefährlichste: Eine Längenprüfung gegen null, bei der die Länge vorzeichenlos ist, kann nie zutreffen. Der Übersetzer weiß das und entfernt sie — und die Prüfung, die den Puffer schützen sollte, ist fort.

Die Regel für die Typwahl

Zweck	Typ	
Größe, Index	`size_t`	**immer** ≥ 0
Zahl mit Vorzeichen	`int32_t`	feste Breite
Bitmuster	`uint32_t`	Umlauf definiert
Zeitpunkt	`int64_t`	nicht `time_t`
Zählwert	`int64_t`	

Die Grundregel: vorzeichenlos nur für Größen und Bitmuster, sonst vorzeichenbehaftet mit fester Breite. Und nie beide in einem Ausdruck mischen — wo es unvermeidlich ist, wandelt man ausdrücklich um und prüft vorher.

Gleitkomma für Messwerte

	`double`	Ganzzahl mit Exponent
$0{,}1 + 0{,}2$	$\neq 0{,}3$	$= 0{,}3$
Vergleich auf Gleichheit	**unbrauchbar**	exakt
Wertebereich	riesig	begrenzt
Geschwindigkeit	Hardware	etwas langsamer
Stellenzahl	nicht darstellbar	**Teil des Werts**

Die letzte Zeile ist der klinische Punkt: Ein Kaliumwert von 4,20 ist etwas anderes als 4,2 — die Stellenzahl sagt, wie genau gemessen wurde. In `double` geht diese Information verloren, und beim Anzeigen wird sie erfunden.

Die Darstellung, die beides erhält.

```
typedef struct { int64_t betrag; int8_t exp; }
Dezimal;
/* 4.20 mmol/l -> { 420, -2 } */
```

Zwei Felder, neun Byte, und der Wert ist exakt darstellbar samt Stellenzahl. Vergleiche werden zu Ganzzahlvergleichen nach Angleichung der Exponenten — exakt und schnell.

Wo `double` richtig ist

Anwendung	`double`**?**	
Kurvenberechnung	**ja**	**Näherung ohnehin**
Statistik	ja	
Signalverarbeitung	ja	
Gespeicherter Messwert	**nein**	Stellenzahl
Geldbetrag	**nein**	

Die Regel ist nicht „Gleitkomma ist schlecht“. Sie lautet: Gleitkomma ist richtig für Größen, die ohnehin nur genähert sind, und falsch für Größen, deren Dezimaldarstellung fachliche Bedeutung hat. Ein Messwert wird gespeichert wie gemessen und gerechnet wie nötig.

Aufgabe 8.1.

(a) Verfolgen Sie einen Rundungsfehler.
(b) Rechnen Sie mit Dezimalwerten.
(c) Bestimmen Sie, wie man Gleitkommazahlen vergleicht.

Lösung Schritt für Schritt.

(a) Ein Rundungsfehler.
Der Fall. Ein Gleitmittel über sechzig Messwerte, fortlaufend geführt.

```
summe += neu;
summe -= alt;          /* der Wert, der herausfaellt
*/
mittel = summe / 60;
```

Nach	Fehler	Bei Wert 4,2	
10^3 Schritten	10^{-13}	unsichtbar	
10^6	10^{-10}	unsichtbar	
10^8	**10^{-8}**	4,20000001	
10^{10}	**10^{-6}**	**4,200001**	sichtbar

Bei fünfhundert Messwerten je Sekunde sind 10^{10} Schritte etwa acht Monate Dauerbetrieb — die Laufzeit eines Überwachungsgeräts zwischen zwei Wartungen. Der Fehler wächst, weil sich Rundungsfehler beim fortlaufenden Addieren und Subtrahieren aufsummieren.

Die drei Abhilfen.

Weg	Aufwand	Fehler	
Regelmäßig neu summieren	gering	begrenzt	**alle 10^4**
Kompensierte Summation	mittel	≈ 0	
Ganzzahlig rechnen	gering	**exakt** 0	**hier richtig**

Bei Messwerten mit fester Stellenzahl ist die dritte Zeile die einfachste und die genaueste: In Einheiten der letzten Stelle gerechnet, ist jede Summe exakt — solange der Ganzzahlbereich reicht, und bei 64 Bit reicht er.

(b) Mit Dezimalwerten rechnen.

```
static bool dez_angleiche(Dezimal *a, Dezimal *b) {
while (a->exp > b->exp) {
if (a->betrag > INT64_MAX / 10) return false;
a->betrag *= 10; a->exp-;
}
while (b->exp > a->exp) {
if (b->betrag > INT64_MAX / 10) return false;
b->betrag *= 10; b->exp-;
}
return true;
}
```

Die Überlaufprüfung vor der Multiplikation ist der Kern — nicht danach, denn danach ist es undefiniert. Die Funktion meldet Fehlschlag, statt still ein falsches Ergebnis zu liefern; das ist der Unterschied aus Kapitel 5.

Die Grenzfälle.

Eingabe	Erwartet	Ergebnis
$\{420, -2\}$ und $\{42, -1\}$	**gleich**	richtig
$\{0, -2\}$ und $\{0, -5\}$	gleich	richtig
$\{10^{18}, 0\}$ und $\{1, -3\}$	**Fehlschlag**	gemeldet
Negative Beträge	vorzeichenrichtig	richtig

Die erste Zeile ist die eigentliche Aufgabe: 4,20 *und* 4,2 *sind derselbe Wert bei verschiedener Genauigkeit. Ein Vergleich muss das erkennen — und die Stellenzahl trotzdem erhalten, wenn er den Wert weitergibt.*

(c) Gleitkommazahlen vergleichen.

Vergleich	Tauglich?	
`a == b`	**fast nie**	
`fabs(a-b) < 1e-9`	bei kleinen Werten	versagt bei großen
Relativ zur Größe	**meist**	$\lvert a-b\rvert \leq \varepsilon \max(\lvert a\rvert, \lvert b\rvert)$
Beides kombiniert	**ja**	auch nahe null

Die zweite Zeile ist die verbreitete Antwort und falsch für große Werte: Bei 10^{12} *ist der Abstand zweier benachbarter* `double` *bereits größer als* 10^{-9} *— die Bedingung kann dann nie zutreffen, außer bei Gleichheit.*

Die Sonderwerte.

Wert	Verhalten	
`NaN`	**ungleich sich selbst**	`x != x` **prüft darauf**
$+0$ und -0	gleich	aber verschiedene Bits
Unendlich	vergleichbar	

Die erste Zeile bricht die Sortierung: Ein Vergleichsverfahren, das `NaN` enthält, kann eine unsinnige Ordnung erzeugen und in manchen Umsetzungen sogar über das Feldende laufen. Deshalb gehören `NaN`-Werte aussortiert, bevor sortiert wird.

Die Umwandlung an der Grenze

```
double d = lies_messwert();
int32_t i = (int32_t)d;    /* undefiniert, wenn d
zu gross */
```

Fall	Verhalten	
Im Bereich	abgeschnitten	**nicht gerundet**
Zu groß	**undefiniert**	
`NaN`	undefiniert	
Negativ, knapp	Richtung null	$-0{,}9 \to 0$

Die erste Zeile ist die häufigste Überraschung: Die Umwandlung schneidet ab, sie rundet nicht. Ein Messwert von 4,9 wird zu 4. Wer runden will, muss es hinschreiben — und vorher prüfen, ob der Wert in den Zielbereich passt.

Goldberg[8] erklärt die Gleitkommaarithmetik so gründlich, dass man sie danach nicht mehr falsch verwendet; van der Linden[26] sammelt die Umwandlungsfallen von C; der Standard[12] legt fest, wo genau die Umwandlung undefiniert wird.

9 Der Übersetzer

Was zwischen Quelltext und Maschine liegt

Stufe	Wandelt	Verändert
Vorverarbeiter	Text	**Makros, Einschlüsse**
Übersetzer vorn	Text zu Zwischenform	
Optimierer	**Zwischenform**	**umfangreich**
Übersetzer hinten	zu Befehlen	Registerzuteilung
Prozessor	**Befehle**	**Reihenfolge, Spekulation**

Fünf Stufen, von denen zwei den Code erheblich umbauen. Der Satz „C ist portabler Assembler“ setzt voraus, dass es diese Stufen nicht gibt — er war 1978 näherungsweise richtig und ist heute irreführend.

Was der Optimierer darf.

Umbau	Erlaubt?	
Reihenfolge ändern	ja	**wenn Ergebnis gleich**
Berechnungen entfernen	ja	unbenutzt
Schleifen ausrollen	ja	
Funktionen einbetten	ja	übergreifend
Prüfungen entfernen	**ja**	**wenn UB angenommen**
Speicherzugriffe zusammenfassen	ja	ohne `volatile`

Die Regel lautet: Alles ist erlaubt, was das beobachtbare Verhalten eines regelkonformen Programms erhält. Wo das Programm nicht regelkonform ist, gibt es kein beobachtbares Verhalten, das zu erhalten wäre — Kapitel 7.

Warnungen als Werkzeug

Einstellung	Findet	
Voreinstellung	wenig	**unbrauchbar**
`-Wall -Wextra`	viel	**Mindestmaß**
`-Wconversion`	stille Umwandlungen	Kap. 8
`-Wswitch-enum`	fehlende Fälle	Kap. 5
`-Werror`	**erzwingt Behebung**	

Die letzte Zeile ist die entscheidende Entscheidung: Warnungen, die nur Warnungen bleiben, werden nach zwei Wochen ignoriert — dieselbe Alarmermüdung wie in Band 18. Wer sie zu Fehlern macht, hält den Bestand sauber; wer es später tut, steht vor tausend Meldungen.

Der Vorverarbeiter

Eigenschaft	Folge	
Arbeitet auf Text	**kennt keine Typen**	keine Prüfung
Ersetzt vor der Übersetzung	Fehler zeigen auf die Makrozeile	schwer zu lesen
Argumente mehrfach eingesetzt	**Nebenwirkungen doppelt**	
Kein Gültigkeitsbereich	Namenskollisionen	

```
#define MAX(a,b) ((a) > (b) ?  (a) :  (b))
int x = MAX(i++, j);    /* i wird zweimal erhoeht */
```

Die Klammern sind richtig gesetzt — und der Fehler bleibt. Ein Makroargument mit Nebenwirkung wird so oft ausgewertet, wie es im Makro vorkommt. In C23 gibt es dafür `auto` und `typeof`; bis dahin gilt: Wo eine `inline`-Funktion genügt, kein Makro.

`volatile` bedeutet weniger, als man denkt

Zusicherung	Gilt?	
Zugriff wird nicht entfernt	**ja**	**Hardware-register**
Reihenfolge unter `volatile`	ja	
Atomar	**nein**	
Sichtbar für andere Fäden	**nein**	**häufiger Irrtum**
Ersetzt Speicherbarrieren	nein	

`volatile` ist für Speicher gedacht, der sich außerhalb des Programms ändert — Hardwareregister, Signalbehandlung. Für Nebenläufigkeit taugt es nicht; dafür gibt es `_Atomic`. Der Irrtum in der vierten Zeile hat viele schwer auffindbare Fehler erzeugt.

Aufgabe 9.1.

(a) Lesen Sie, was der Übersetzer erzeugt.
(b) Bestimmen Sie die Wirkung der Optimierungsstufen.
(c) Untersuchen Sie, wann man den Übersetzer bremsen muss.

Lösung Schritt für Schritt.

(a) Was der Übersetzer erzeugt.
Der Fall. Die Grenzprüfung aus dem Messdatenerfasser.

```
bool ausserhalb(int32_t w, int32_t lo, int32_t hi) {
return w < lo || w > hi;
}
```

Stufe	Befehle	Sprünge	Dauer
`-O0`	21	3	4,1 ns
`-O1`	7	2	1,2 ns
`-O2`	**4**	**0**	**0,4** ns

Bei `-O2` verschwinden die Sprünge: Der Übersetzer rechnet beide Vergleiche und verknüpft die Ergebnisse, statt zu verzweigen. Das ist schneller, weil eine falsch vorhergesagte Verzweigung mehr kostet als zwei überflüssige Vergleiche.

Bemerkenswert ist der Faktor zehn zwischen `-O0` und `-O2`. Wer Laufzeiten misst, muss mit derselben Stufe messen, mit der ausgeliefert wird — Messungen unter `-O0` sind wertlos.

(b) Die Optimierungsstufen.

Stufe	Geschwind.	Übersetzungs-zeit	Fehlersuche
`-O0`	1,0	1,0	**gut**
`-O1`	2,8	1,6	brauchbar
`-O2`	**3,4**	2,4	mühsam
`-O3`	3,5	3,1	schwer
`-Os`	2,9	2,2	**kleinster Code**

Der Sprung von `-O2` auf `-O3` bringt drei Prozent und kostet dreißig Prozent Übersetzungszeit — deshalb ist `-O2` fast überall die richtige Wahl. Bei eingebetteten Systemen gewinnt `-Os`, weil dort der Programmspeicher knapp ist.

Die Kombination, die man tatsächlich braucht.

Entwicklung	`-O1 -g -Wall -Wextra -Werror` und Sanitizer
Prüfstrecke	**Beides: mit und ohne Sanitizer, `-O2`**
Auslieferung	`-O2 -g` **— Symbole behalten, nicht entfernen**

Die letzte Zeile ist ein häufig unterlassener Rat: Symbolinformationen kosten Plattenplatz und keine Laufzeit. Ohne sie ist ein Absturzbericht aus dem Betrieb eine Liste von Adressen ohne Bedeutung.

(c) Wann man den Übersetzer bremsen muss.

Lage	Mittel	
Hardwareregister	`volatile`	
Nebenläufiger Zugriff	`_Atomic`	**nicht `volatile`**
Reihenfolge erzwingen	Barrieren	
Geheimnis löschen	**`memset_explicit`**	**sonst entfernt**
Funktion nicht einbetten	`noinline`	selten nötig

Die vierte Zeile ist ein Lehrstück: Ein `memset`, das einen Schlüsselpuffer vor dem Freigeben löscht, wird vom Optimierer entfernt — der Puffer wird ja nicht mehr gelesen. Der Übersetzer hat recht und beseitigt genau die Sicherheitsmaßnahme.

Das ist derselbe Mechanismus wie bei der entfernten Überlaufprüfung: Der Optimierer sieht keinen Zweck, weil der Zweck außerhalb des Sprachmodells liegt. Wo das der Fall ist, muss man es ausdrücklich sagen.

Der Prozessor ordnet auch um

Ebene	Ordnet um	
Übersetzer	ja	**beim Bauen**
Prozessor	**ja**	**zur Laufzeit**
Zwischenspeicher	ja	Sichtbarkeit verzögert

Auch wenn man den Übersetzer bändigt, ordnet der Prozessor Speicherzugriffe um. Zwei Fäden können deshalb eine unterschiedliche Reihenfolge derselben Schreibvorgänge sehen — ohne dass ein Fehler im Code steht.

Deshalb ist die einzig belastbare Antwort für nebenläufigen Zugriff das Speichermodell der Sprache: `_Atomic` mit passender Ordnung. Alles andere — `volatile`, Schlafpausen, Glaube — ist unbelegt.

Aho und Mitautoren[1] beschreiben die Stufen der Übersetzung im Einzelnen; Lattner[18] zeigt, welche Schlüsse der Optimierer aus dem Standard zieht; Fog[7] misst, was daraus an Maschinenbefehlen wird.

Part IV

Das Handwerk

10 Werkzeuge

Was welches Werkzeug findet

Werkzeug	Wann	Findet
Übersetzer-warnungen	**beim Bauen**	sofort, kostenlos
Statische Analyse	beim Bauen	**Pfade ohne Ausführung**
Sanitizer	**beim Laufen**	**tatsächliche Verstöße**
Zufallsprüfung	beim Laufen	**unerwartete Eingaben**
Speicherprüfer	beim Laufen	ohne Neuübersetzung

Die Reihenfolge ist auch die Reihenfolge der Einführung: von oben nach unten steigt der Aufwand und sinkt die Zahl der übersehenen Fälle. Wer nur eines einführen kann, nimmt die dritte Zeile — sie findet die Klassen aus Kapitel 7.

Statisch gegen dynamisch.

	Statisch	Sanitizer
Sieht	**alle Pfade**	**ausgeführte Pfade**
Irrt in Richtung	Fehlalarm	übersieht
Braucht	nichts	Prüffälle
Kosten	Übersetzungs-zeit	Laufzeit

Beide irren nur in eine Richtung, und die Richtungen sind entgegengesetzt — deshalb ergänzen sie einander, statt sich zu ersetzen. Dasselbe Argument wie in Band 17, Kapitel 8, hier mit stärkerer Wirkung, weil C mehr zu finden gibt.

Zufallsprüfung

1	Eine Funktion nimmt einen Bytepuffer entgegen.
2	Das Werkzeug erzeugt Eingaben und beobachtet die Ausführung.
3	**Es bevorzugt Eingaben, die neue Codepfade erreichen.**
4	Bei Absturz oder Sanitizer-Meldung: Fall gespeichert und verkleinert.

Schritt 3 macht den Unterschied zum blinden Zufall: Das Werkzeug lernt aus der Abdeckung, welche Eingaben interessant sind, und findet damit in Stunden Pfade, die eine Prüfsuite nie erreicht. Schritt 4 ist das Verkleinern aus Band 20, Kapitel 7 — hier automatisch.

Der Ertrag am Beispiel

Der Fall. Der Zeilenzerleger aus Kapitel 4, 70 Zeilen.

Verfahren	Aufwand	Gefunden	Neu
Durchsicht	2 h	2	2
Prüffälle von Hand	3 h	4	2
Sanitizer dazu	0,5 h	6	2
Zufallsprüfung, 4 h	**1 h**	**11**	**5**

Die letzte Zeile findet mehr als alle anderen zusammen — bei einer Stunde Einrichtung und vier Stunden Rechenzeit über Nacht. Die fünf zusätzlichen Fälle waren alle Eingaben, an die niemand gedacht hätte: leere Zeile mit Trennzeichen, Zeile ohne Zeilenende, 65 000 Trennzeichen hintereinander.

Was die Einrichtung kostet.

```
int LLVMFuzzerTestOneInput(const uint8_t *d, size_t n)
{
Str felder[5];
zerlege((Str){ (const char *)d, n }, felder, 5);
return 0;
}
```

Fünf Zeilen. Das ist der ganze Aufwand für eine Prüfung, die über Nacht Millionen Eingaben durchprobiert. Der Grund, warum es selten gemacht wird, ist nicht der Aufwand — es ist Unkenntnis.

Ein Werkzeug, das nicht läuft, findet nichts

Zustand	Wirkung	
Eingerichtet, läuft nie	**keine**	**häufigster Fall**
Läuft, Meldungen ignoriert	keine	Alarmermüdung
Läuft in der Prüfstrecke	**volle**	**erzwungen**

Heartbleed wäre von einem Speicherprüfer beim ersten Lauf gemeldet worden. Das Werkzeug gab es seit Jahren. Die Lücke lag nicht im Werkzeugbestand, sondern darin, dass es nicht Teil des Bauvorgangs war — die Lehre aus dem Vorwort.

Aufgabe 10.1.

(a) Richten Sie eine Prüfstrecke ein.

(b) Bewerten Sie Fehlalarme der statischen Analyse.

(c) Bestimmen Sie den Nutzen einer Kodierrichtlinie.

Lösung Schritt für Schritt.

(a) Eine Prüfstrecke.

St.	Schritt	Dauer	Wann
1	Bauen mit `-Werror`	40 s	**jede Übergabe**
2	Prüfungen normal	12 s	jede Übergabe
3	**Prüfungen mit ASan/UBSan**	28 s	**jede Übergabe**
4	Statische Analyse	3 min	jede Übergabe
5	Prüfungen mit TSan	2 min	nächtlich
6	**Zufallsprüfung**	4 h	**nächtlich**

Die ersten vier Stufen dauern zusammen gut vier Minuten — innerhalb der Grenze aus Band 17, Kapitel 10, ab der niemand mehr wartet. Die teuren Prüfungen laufen nachts, und ihr Befund liegt am Morgen vor.

Einrichtung	Aufwand	
Einmalig	≈ 16 h	
Altlastmeldungen	≈ 400	**die Hürde**
Vorgehen	**einfrieren**	**nur Neues prüfen**

Die zweite Zeile ist der Grund, warum solche Einführungen scheitern. Der Ausweg ist derselbe wie in Band 19: Altbestand einfrieren, für neuen Code streng prüfen — und die Altlast bei jeder Berührung ein Stück abbauen.

(b) Fehlalarme bewerten.

Strenge	Meldungen	davon echt	Genauigkeit
Hoch	340	28	**8** %
Mittel	95	23	24 %
Niedrig	**31**	**19**	**61** %

Bei acht Prozent Genauigkeit lernt man, Meldungen wegzuklicken — und übersieht dann auch die achtundzwanzig echten. Bei einundsechzig Prozent nimmt man jede ernst. Neunzehn gefundene Fehler bei hoher Verlässlichkeit sind mehr wert als achtundzwanzig in einem Rauschen.

Das ist genau der Befund aus Band 19, Kapitel 4. Er gilt hier verschärft, weil die statische Analyse von C wegen der Zeigerarithmetik besonders viele Fehlalarme erzeugt.

(c) Nutzen einer Kodierrichtlinie.

Regel	Wirkung	Prüfbar
Kein `goto` außer Aufräumen	gut	ja
Keine Rekursion	**Kellergrenze**	ja
Keine dynamische Belegung	**vorhersagbar**	ja
Alle Pfade geben Wert zurück	gut	ja
Ein Rückgabepunkt	**umstritten**	ja

Die zweite und dritte Zeile sind für eingebettete Systeme entscheidend und anderswo zu streng: Ohne dynamische Belegung ist der Speicherbedarf zur Übersetzungszeit bekannt — damit kann ein Messdatenerfasser nicht an Speichermangel scheitern.

Die letzte Zeile widerspricht dem Aufräummuster aus Kapitel 5 scheinbar und meint dasselbe: ein Rückgabepunkt, erreicht über Vorwärtssprünge. Solche Regeln muss man lesen, statt sie zu zitieren.

Richtlinien ersetzen kein Urteil

Erwartung	Erfüllt?	
Fehlerklassen ausgeschlossen	**teilweise**	
Nachweis für Zulassung	ja	**regulatorisch**
Guter Entwurf	**nein**	
Ersetzt Werkzeuge	nein	ergänzt sie

Eine Kodierrichtlinie schließt Fehlerklassen aus und sagt nichts über Zerlegung, Namensgebung oder Datenmodell — also über alles, was Band 20 behandelt. Regelkonformer Code kann unlesbar sein, und unlesbarer Code ist auf Dauer unsicher.

Die Werkzeugdokumentation[27] beschreibt, was die einzelnen Prüfer erfassen; Seacord[23] ordnet sie den Fehlerklassen zu; die MISRA-Richtlinie[20] zeigt, wie weit man C durch Regeln einengen kann.

11 Module und Schnittstellen

Was C an Kapselung bietet

Mittel	Verbirgt	
Kopfdatei	**nichts — zeigt**	Schnittstelle
`static`	**Funktion, Variable**	aus der Übersetzungseinheit
Unvollständiger Typ	**den Aufbau**	**echte Kapselung**
Getrennte Übersetzung	Abhängigkeiten	

Die dritte Zeile ist das stärkste Mittel und wird selten genutzt: Ein Typ, dessen Aufbau nur in der Umsetzungsdatei steht, kann von außen weder gelesen noch angelegt werden. Das ist Parnas' Geheimnisprinzip aus Band 17, in C erhältlich.

Der unvollständige Typ.

```
/* liste.h -- was der Benutzer sieht */
typedef struct Liste Liste;
Liste *liste_erzeuge(size_t anfang);
void   liste_frei(Liste *l);
int    liste_anfuege(Liste *l, const Befund *b);
size_t liste_anzahl(const Liste *l);

/* liste.c -- was verborgen bleibt */
struct Liste { Befund *daten; size_t n, kap; };
```

	Offen	Unvollständig
Zugriff von außen	**ja**	**nein**
Aufbau änderbar	nein	**ja**
Auf dem Keller anlegbar	ja	nein
Kosten	0	eine Belegung

Der Preis steht in der dritten Zeile: Ein unvollständiger Typ muss auf der Halde liegen, weil der Aufrufer seine Größe nicht kennt. Für einen Messdatenerfasser ohne dynamische Belegung ist das ein Ausschlusskriterium — dort nimmt man den offenen Typ und die Vereinbarung, ihn nicht anzufassen.

Was in eine Kopfdatei gehört

Inhalt	Gehört hinein?	
Öffentliche Funktionen	**ja**	
Öffentliche Typen	ja	
Vertrag als Kommentar	**ja**	**Besitz, Nullzeiger**
Umsetzungsdetails	**nein**	
Andere Kopfdateien	nur nötige	Bauzeit
`static`-Funktionen	nein	

Die dritte Zeile ist das, was C an Verträgen zu bieten hat — Band 17, Kapitel 6, ohne Prüfung. Wer schreibt, ob ein Nullzeiger erlaubt ist und wer freigibt, ersetzt keine Zusicherung und beseitigt die häufigste Unklarheit.

Was Einschlüsse kosten.

Aufbau	Zeilen je Einheit	Bauzeit	
Alles einschließen	≈ 180 000	**94 s**	
Nur Nötiges	≈ 24 000	**19 s**	5×
Vorwärts-deklarationen	≈ 11 000	12 s	

Faktor fünf bei der Bauzeit — und die Bauzeit bestimmt, wie oft jemand baut. Das ist die Rückmeldedauer aus Band 17, Kapitel 10: Bei neunzig Sekunden schaut man weg, bei zwanzig wartet man ab.

Die Faustregel für Einschlüsse

Verwendung	Nötig
Nur Zeiger auf `T`	**Vorwärtsdeklaration**
`T` als Feld oder Wert	**vollständiger Typ**
Nur in der `.c`-Datei	**dort einschließen**

Ein `struct Patient;` in der Kopfdatei genügt, solange nur Zeiger vorkommen. Das entkoppelt Module und verkürzt die Bauzeit — und es zwingt dazu, die Frage zu beantworten, ob man den Aufbau wirklich braucht.

Aufgabe 11.1.

(a) Schneiden Sie ein Modul zu.
(b) Bestimmen Sie die Abhängigkeitsrichtung.
(c) Untersuchen Sie, was `static` bewirkt.

Lösung Schritt für Schritt.

(a) Ein Modul zuschneiden.
Der Bestand. Der Messdatenerfasser, 4200 Zeilen in einer Datei.

Modul	Zeilen	Öffentlich	Prüfbar
Gerätezugriff	780	4	Hardware
Umrechnung	340	2	**rein**
Grenzprüfung	**620**	3	**rein**
Ringpuffer	410	6	rein
Übertragung	890	3	Attrappe
Ablaufsteuerung	560	2	mittel
Protokollierung	600	2	

Drei Module mit zusammen 1370 *Zeilen sind reine Rechnung — ohne Hardware, ohne Zeit, ohne Speicherbelegung. Sie lassen sich auf einem gewöhnlichen Rechner prüfen, nicht nur auf dem Zielgerät. Das ist ein Drittel des Bestands und der Teil, in dem die fachlichen Fehler sitzen.*

Die Wirkung.

	Vorher	Nachher
Prüfung auf dem Zielgerät	**alles**	≈ 60 %
Prüfdauer je Lauf	**4** min	**6** s
Prüffälle	12	**140**
Öffentliche Namen	≈ 90	**22**

Die zweite Zeile erklärt die dritte: Bei vier Minuten je Lauf schreibt niemand hundertvierzig Fälle. Die Prüfbarkeit entscheidet über die Zahl der Prüfungen, und die Zerlegung entscheidet über die Prüfbarkeit.

(b) Abhängigkeitsrichtung.

Modul	hängt von	Ebene	Zulässig?
Ablaufsteuerung	5	**oben**	ja
Übertragung	2	mitte	ja
Grenzprüfung	**0**	**unten**	**ja**
Protokollierung	0	unten	ja
Ringpuffer	**1**	unten	**prüfen**

Die letzte Zeile ist der Fund: Der Ringpuffer hängt von der Protokollierung ab, weil er Überläufe meldet. Damit kann man ihn nicht ohne Protokollierung prüfen. Der Ausweg ist eine Rückrufadresse — der Puffer meldet an den, der ihn erzeugt hat.

Die Regel: Abhängigkeiten zeigen nach unten, nie zur Seite und nie nach oben. Wer sie einhält, kann jedes Modul einzeln prüfen — und die Prüfung braucht nur die Ebenen darunter.

(c) Was `static` bewirkt.

Wirkung	Bei Funktionen	Bei Variablen
Sichtbarkeit	**nur in der Datei**	nur in der Datei
Optimierung	**einbettbar**	
Lebensdauer	—	**das ganze Programm**
Nebenläufig	—	**gefährlich**

`static` bedeutet zwei verschiedene Dinge, je nachdem wo es steht — eine Eigenheit der Sprache. Bei einer Funktion ist es fast immer richtig; bei einer Variable innerhalb einer Funktion erzeugt es verborgenen Zustand mit allen Folgen aus Band 20, Kapitel 6.

Die Wirkung auf die Optimierung.

	Ohne `static`	Mit
Einbettung	nur mit LTO	**immer**
Aufrufkosten	≈ 2 ns	**0**
Ungenutzt entfernbar	nein	**ja**

Die dritte Zeile ist ein stiller Gewinn: Eine nicht benutzte `static`-Funktion meldet der Übersetzer als Warnung — toter Code wird sichtbar. Bei einer öffentlichen Funktion kann er das nicht wissen, und der tote Code bleibt jahrelang liegen.

Die Kopfdatei ist der Vertrag

Änderung	Bricht?	
Funktion hinzufügen	nein	
Argument hinzufügen	**ja**	alle Aufrufer
Feld im offenen Typ	**ja**	**auch binär**
Feld im unvollständigen Typ	**nein**	der Gewinn

Die dritte Zeile ist bei Bibliotheken folgenschwer: Ein zusätzliches Feld ändert die Größe der Struktur, und ein Programm, das gegen die alte Fassung gebunden wurde, schreibt daneben. Solche Fehler treten erst beim Kunden auf und sind kaum zu diagnostizieren.

Hanson[10] zeigt an durchgearbeiteten Beispielen, wie Schnittstellen in C aussehen sollten; Gustedt[9] behandelt die heutigen Sprachmittel dafür; Kernighan und Pike[13] verbinden Schnittstellenentwurf mit dem übrigen Handwerk.

12 Fehlersuche in C

Was hier anders ist

	In Python	In C
Fehlerort	**wo er auftritt**	**oft ganz woanders**
Meldung	Ausnahme mit Text	Absturz oder gar nichts
Reproduzierbar	meist	**oft nicht**
Zustand lesbar	ja	mit Symbolen

Die erste Zeile ist die eigentliche Schwierigkeit: Ein Schreibzugriff über eine Feldgrenze beschädigt eine andere Variable, und das Programm stürzt Minuten später an einer Stelle ab, die mit dem Fehler nichts zu tun hat. Die Ablaufverfolgung zeigt dann den Ort des Symptoms.

Die sieben Schritte, angepasst.

	Wie in Band 20	Zusätzlich in C
1	Reproduzieren	**Sanitizer einschalten**
2	Verkleinern	
3	Beobachten	Speicherinhalt ansehen
4	Hypothese	
5	Widerlegende Vorhersage	
6	Prüfen	
7	Ursache benennen	**nicht das Symptom**

Nur zwei Ergänzungen — das Verfahren aus Band 20 gilt unverändert. Der erste Schritt ist in C aber wörtlich zu nehmen: Ein Sanitizer verwandelt einen lautlosen Fehler in einen lauten und verschiebt damit den Fundort an den Ort der Ursache.

Der Sanitizer als Ortungsgerät

	Ohne	Mit ASan
Meldung bei	späterem Absturz	**dem Zugriff**
Abstand zur Ursache	Minuten, Aufrufe	**null**
Ausgabe	Adresse	**Zeile, Feld, Abstand**
Zeit bis Fund	≈ 6 h	**≈ 5 min**

Der Prüfer meldet nicht nur den Zugriff, sondern auch, welches Feld getroffen wurde, wie weit daneben, wo es angelegt und gegebenenfalls freigegeben wurde. Aus einer Adressensuche wird ein Bericht — und aus sechs Stunden fünf Minuten.

Der Fall: eine Warnung zu viel

Die Meldung. Der Messdatenerfasser gibt gelegentlich eine Grenzwertwarnung für einen Wert aus, der im Bereich liegt — etwa alle zwei Tage.

Reproduzierbar	Nein — alle zwei Tage einmal
Erste Vermutung	Rundungsfehler in der Umrechnung
Warum falsch	**Rundung erklärt keine Zweitagesperiode**

Die dritte Zeile ist der methodische Kern: Eine Hypothese muss nicht nur den Fehler erklären, sondern auch seine Häufigkeit. *Rundung wirkt bei jedem Wert oder bei keinem — nicht alle zwei Tage.*

Die Suche nach der Periode.

Größe	Periode	Passt?
Messwerte	500/s	nein
Ringpuffer voll	alle 4 min	nein
Protokolldatei rollt	täglich	knapp
Zähler läuft über	**alle 49,7 Tage**	nein
2^{32} Zehntelms	**alle 2,0 Tage**	**ja**

Ein Zeitstempel in Zehntelmillisekunden läuft nach 2^{32} Einheiten über — das sind $4{,}97 \cdot 10^5$ Sekunden, also knapp sechs Tage; bei vorzeichenbehafteter Deutung die Hälfte. Beim Überlauf wird die Zeitdifferenz negativ, und die Steigungsprüfung schlägt an.

Die Periode war die Spur, nicht der Code. Wer bei einem unregelmäßigen Fehler zuerst nach einer Periode sucht, hat oft schon die halbe Antwort — und das ist eine Technik, die kein Werkzeug ersetzt.

Der Fehler, der beim Hinsehen verschwindet

Ursache	Verschwindet bei	
Nebenläufigkeit	**Haltepunkt**	Zeitverhalten
Uninitialisierter Speicher	**Prüf-übersetzung**	Halde genullt
Kellerüber-schreibung	anderer Aufbau	
Zeigerfehler	Ausgabezeile eingefügt	**Anordnung verschoben**

Die zweite Zeile ist die häufigste Überraschung: Unter `-O0` wird der Speicher oft mit Mustern gefüllt, im Betrieb nicht. Ein Programm, das nur in der Auslieferungsfassung falsch läuft, hat fast immer eine uninitialisierte Variable — und MSan findet sie in Minuten.

Die Regel daraus: Wenn ein Fehler beim Beobachten verschwindet, ist das Verschwinden selbst der wichtigste Befund. Es sagt, welche Fehlerklasse vorliegt — und damit, welches Werkzeug man braucht.

Aufgabe 12.1.

(a) Lesen Sie einen Absturzbericht.
(b) Finden Sie ein Speicherleck.
(c) Bestimmen Sie, wie man im Betrieb sucht.

Lösung Schritt für Schritt.

(a) Einen Absturzbericht lesen.

Angabe	Bedeutet	
Signal 11	**ungültiger Zugriff**	
Adresse 0	**Nullzeiger**	leicht
Adresse klein	Nullzeiger plus Feld	leicht
Adresse 0x`deadbeef`	**Muster**	**uninitialisiert**
Adresse plausibel	**freigegeben oder daneben**	schwer

Die Adresse ist die erste Information, nicht die Ablaufverfolgung. Eine kleine Adresse bedeutet fast immer einen Nullzeiger, auf den ein Feldzugriff addiert wurde — und der Abstand sagt, welches Feld.

Wenn die Verfolgung unsinnig aussieht.

Bild	Deutung	
Funktionen, die nicht aufrufen	**Keller überschrieben**	Pufferfehler
Verfolgung bricht ab	Rahmenzeiger fort	Optimierung
Nur Adressen	**keine Symbole**	**Kap. 9**

Die erste Zeile ist ein starker Befund: Wenn die Aufrufkette unmöglich ist, wurde der Keller beschädigt — und damit ist die Fehlerklasse bekannt, bevor man eine Zeile Code gelesen hat.

(b) Ein Speicherleck finden.
Das Bild. Der Erfasser wächst um etwa 40 MB je Tag.

Schritt	Werkzeug	Ergebnis
Wachstum bestätigen	Betriebszähler	40 MB/Tag
Größenklasse	Prüfer	≈ 2,9 **Mio. Blöcke à** 14 **B**
Herkunft	**Leckprüfer**	**eine Aufrufstelle**
Ursache	Lesen	Fehlerpfad ohne Freigabe

Die zweite Zeile grenzt bereits stark ein: Vierzehn Byte je Block deutet auf eine kleine Struktur, und die Zahl entspricht der Zahl verworfener Messwerte. Damit ist der Verdacht auf den Fehlerpfad gerichtet, bevor ein Werkzeug lief.

Der Fehlerpfad ist der klassische Ort für Lecks — er wird selten durchlaufen und deshalb selten geprüft. Das Aufräummuster aus Kapitel 5 hätte ihn ausgeschlossen.

(c) Suchen im Betrieb.

Mittel	Im Betrieb?	
Schrittweise Verfolgung	**nein**	**hält das Gerät an**
Sanitizer	UBSan ja	≈ 20 % Laufzeit
Abbild bei Absturz	**ja**	**das wichtigste Mittel**
Strukturierte Protokolle	ja	Band 20, Kap. 7
Zähler und Messpunkte	ja	billig

Ein Absturzabbild mit Symbolen ist bei einem Gerät, das man nicht anhalten kann, das einzige, was man bekommt. Es einzurichten kostet einen Tag und entscheidet, ob ein Absturz beim Kunden auswertbar ist oder nicht.

Was in das Abbild gehört.

Inhalt	Nötig?	
Aufrufkeller	**ja**	
Registerinhalte	ja	
Fassungsnummer	**ja**	**sonst falsche Symbole**
Letzte Protokollzeilen	ja	
Patientendaten	**nein**	**Band 19**

Die letzte Zeile ist die Regel aus Band 21, Kapitel 11 und Band 19: Ein Abbild wandert an den Hersteller. Was an Patientendaten im Speicher stand, darf nicht mitgehen — und das ist bei einem vollständigen Speicherabbild schwer sicherzustellen. Deshalb: gezieltes Abbild statt vollständigem.

Die Ursache liegt vor dem Symptom

Symptom	Ort	Ursache
Absturz in `free`	Speicher-verwaltung	**Überschrei-bung anderswo**
Falscher Wert	Anzeige	Umwandlung
Warnung zu viel	Grenzprüfung	**Zeitüberlauf**
Absturz nach Tagen	beliebig	Leck oder Umlauf

Die erste Zeile ist der Klassiker: Ein Absturz innerhalb der Speicherverwaltung bedeutet fast nie einen Fehler dort, sondern eine Beschädigung der Verwaltungsdaten durch einen Schreibzugriff über eine Blockgrenze — irgendwo, irgendwann vorher.

Deshalb gilt die Prüffrage aus Band 20 hier verschärft: Kann ich erklären, warum der Fehler genau jetzt und genau hier auftrat? Wer das nicht kann, hat in C fast sicher nur das Symptom gefunden.

Die Werkzeugdokumentation[27] beschreibt, wie man die Prüfer im Betrieb einsetzt; van der Linden[26] sammelt die Fehlerbilder, die C erzeugt; Kernighan und Pike[13] behandeln Fehlersuche als tägliches Handwerk.

Part V

Das Urteil

13 Der zweite Entwurf in C

Was hier anders zählt

Kriterium	Band 20	Hier zusätzlich
Verstehenszeit	ja	ja
Änderbarkeit	ja	ja
Prüfbarkeit	ja	ja
Größenordnung	ja	ja
Fehlerklassen ausgeschlossen	—	**ja**
Speicherbedarf vorhersagbar	—	**ja**

Zwei zusätzliche Bedingungen. Die fünfte ist die wichtigere: In Python fragt man, ob der Code verständlich ist; in C fragt man zusätzlich, welche Fehlerklassen der Entwurf noch zulässt — und ob man sie durch eine andere Bauform ausschließen kann.

Der Fall: der Ringpuffer

Der erste Entwurf.

```
typedef struct {
Messwert *daten;
int kopf, fuss, anzahl, kap;
} Ring;

void ring_lege(Ring *r, Messwert m) {
r->daten[r->kopf] = m;
r->kopf = (r->kopf + 1) % r->kap;
if (r->anzahl < r->kap) r->anzahl++;
else r->fuss = (r->fuss + 1) % r->kap;
}
```

Elf Zeilen, funktioniert, und enthält fünf Schwächen — von denen keine ein Fehler im engeren Sinn ist. Wir gehen sie durch.

Schwäche	Wirkung	
`int` für Indizes	**Überlauf, Vorzeichen**	**Kap. 8**
Modulo	≈ 20 Takte	je Aufruf
Dynamischer Speicher	**unvorhersagbar**	
Drei Zähler	**redundant**	können widersprechen
Nicht nebenläufig	Wettlauf	

Der zweite Entwurf.

```
#define RING_KAP 4096u       /* Zweierpotenz, fest */

typedef struct {
Messwert daten[RING_KAP];  /* kein malloc */
_Atomic uint32_t kopf, fuss;
} Ring;

static inline bool ring_lege(Ring *r, Messwert m) {
uint32_t k = atomic_load_explicit(&r->kopf,
memory_order_relaxed);
uint32_t f = atomic_load_explicit(&r->fuss,
memory_order_acquire);
if (k - f == RING_KAP) return false;  /* voll */
r->daten[k & (RING_KAP - 1)] = m;
atomic_store_explicit(&r->kopf, k + 1,
memory_order_release);
return true;
}
```

Der Kunstgriff steht in der Vollprüfung: `k - f` rechnet mit vorzeichenlosen Zählern, die frei überlaufen dürfen — die Differenz bleibt richtig, weil der Umlauf definiert ist. Deshalb braucht es keinen dritten Zähler, und zwei Wahrheiten können nicht mehr widersprechen.

Die Bilanz.

	Erster	Zweiter	
Zeilen	11	18	+64 %
Dauer je Ablage	9 ns	**2** ns	4,5×
Speicher	dynamisch	**fest**	
Nebenläufig	nein	**ein Schreiber**	
Fehlerklassen	**5**	**1**	
Bei vollem Puffer	überschreibt still	**meldet**	

Wieder das Muster aus Band 20: Der zweite Entwurf ist länger. Er ist viermal schneller, im Speicherbedarf vorhersagbar und schließt vier Fehlerklassen aus — und die letzte Zeile macht aus einem stillen Datenverlust eine Meldung.

Warum die Zweierpotenz

Rechnung	Takte	
`i % n`	≈ 20	**Division**
`i & (n-1)`	**1**	**nur bei 2^k**

Faktor zwanzig für eine Einschränkung, die nichts kostet — die Puffergröße ist ohnehin frei wählbar. Das ist die Art von Entscheidung, die man beim Entwurf trifft und später nicht mehr nachrüsten kann, ohne die Schnittstelle zu ändern.

Aufgabe 13.1.

(a) Prüfen Sie die Abbruchbedingungen.
(b) Bestimmen Sie, welche Fehlerklasse bleibt.
(c) Untersuchen Sie den Preis der Festgröße.

Lösung Schritt für Schritt.

(a) Die Abbruchbedingungen.

Bedingung	Prüfung	Ergebnis
Verstehenszeit	5 Fragen	**70** s mäßig
Änderbarkeit	andere Größe	eine Konstante
Prüfbarkeit	ohne Hardware	41 **Fälle**
Größenordnung	500/s	2 ns
Fehlerklassen	—	1 verbleibt
Speicher	—	**fest, bekannt**

Fünf von sechs erfüllt. Die Verstehenszeit ist mit siebzig Sekunden schlechter als beim ersten Entwurf — die Speicherordnungen verlangen Wissen, das nicht jeder hat. Das ist ein echter Preis und kein Versehen.

Die Abhilfe ist kein weiterer Umbau, sondern ein Kommentar, der das Protokoll erklärt: ein Schreiber, ein Leser, warum `acquire` und `release` an genau diesen Stellen stehen. Fünf Zeilen Text ersetzen hier fünf Minuten Nachdenken bei jedem Leser.

(b) Welche Fehlerklasse bleibt.

Klasse	Ausge-schlossen?	
Speicherleck	**ja**	**kein `malloc`**
Grenzüber-schreitung	ja	Maske
Überlauf	ja	definiert
Widersprüchliche Zähler	**ja**	**nur zwei**
Mehr als ein Schreiber	**nein**	**Vereinbarung**

Die letzte Zeile ist die verbleibende Schwäche: Der Puffer ist für genau einen Schreiber und einen Leser richtig. Bei zwei Schreibern ist er falsch — und nichts erzwingt das. Es ist eine Vereinbarung, wie der Besitz in Kapitel 3.

Die ehrliche Antwort lautet: aufschreiben und im Namen führen — `ring_ss` für einen Schreiber. Eine Vereinbarung im Namen wird eher bemerkt als eine im Kommentar, und mehr bietet C nicht.

(c) Der Preis der Festgröße.

Größe	Speicher	Pufferzeit	Verlust
512	12 KB	1,0 s	0,4 %
4096	**98 KB**	**8,2 s**	**0,002 %**
16384	393 KB	33 s	≈ 0

Achtundneunzig Kilobyte für acht Sekunden Puffer — genug, um eine Netzstörung zu überbrücken. Der Verlust bei Überlauf sinkt um Faktor zweihundert gegenüber der kleineren Größe, und die nächste Verachtfachung bringt fast nichts mehr.

Der entscheidende Vorteil der Festgröße ist nicht der Speicher, sondern die Gewissheit: Der Bedarf steht zur Übersetzungszeit fest, und das Gerät kann nicht an Speichermangel scheitern. Bei einem Messdatenerfasser ist das keine Optimierung, sondern eine Anforderung.

Wann der zweite Entwurf zu weit geht

Maßnahme	Hier nötig?	
Atomare Zähler	**ja**	**zwei Fäden**
Zweierpotenz	ja	kostenlos
Feste Größe	ja	**Anforderung**
Sperrfrei für n Schreiber	**nein**	**es gibt einen**
Eigene Speicherordnung je Feld	nein	unlesbar

Die vierte Zeile ist die Versuchung: Ein allgemeiner sperrfreier Puffer für beliebig viele Schreiber ist eine reizvolle Aufgabe, hundertfünfzig Zeilen lang und für dieses Gerät nutzlos. Die Regel der drei Fälle aus Band 20 gilt — es gibt einen Fall.

In C ist diese Versuchung größer als anderswo, weil die Sprache tiefe Eingriffe erlaubt und belohnt. Umso wichtiger ist die Abbruchbedingung: erfüllt ist erfüllt, auch wenn mehr möglich wäre.

Klemens[16] zeigt, wie heutiges C aussieht statt des Stils von 1988; Gustedt[9] behandelt die Sprachmittel einschließlich des Speichermodells; King[15] bietet die gründliche Darstellung für alles, was hier vorausgesetzt wird.

14 Wann C und wann nicht

Die vier Gründe, die tragen

Grund	Trägt?	
Harte Zeitschranken	**ja**	**keine Bereinigungspause**
Sehr wenig Speicher	**ja**	Kilobyte statt Megabyte
Keine Laufzeitumgebung	**ja**	**Kern, Firmware**
Vorhandener Bestand	**ja**	der häufigste Grund

Vier Gründe, und der vierte ist in der Praxis der häufigste: Man schreibt C, weil das System in C geschrieben ist. Das ist ein vollständig legitimer Grund und sollte auch so benannt werden — nicht als technische Überlegenheit verkleidet.

Die Gründe, die nicht tragen.

Behauptung	Trägt?	
„C ist schnell“	**unvollständig**	**erst messen**
„Näher an der Maschine“	nein	Kap. 9
„Volle Kontrolle“	teils	auch über Fehler
„Läuft überall“	teils	andere auch
„Keine Abhängigkeiten“	teils	**man baut sie selbst**

Die erste Zeile ist die häufigste und die schwächste: Wenn fünfundfünfzig Prozent der Zeit in Datenbankabfragen vergehen (Band 20, Kapitel 12), ändert die Sprachwahl fünfzehn Prozent der Laufzeit — und kostet vierzehnmal so viele Zeilen.

Die Alternativen

	Zeitschranken	Speichersicher	Bestand
C	ja	**nein**	riesig
C++	ja	teils	groß
Rust	**ja**	**ja**	wachsend
Zig	ja	teils	klein
Ada	ja	**ja**	**Luftfahrt**
Go	**nein**	ja	**Band 22**

Die dritte und fünfte Zeile erfüllen beide harten Bedingungen und schließen die Fehlerklassen aus Kapitel 3 aus. Wer heute ein neues systemnahes Vorhaben beginnt, muss begründen, warum nicht *eine davon — und die Begründung lautet meist: der Bestand.*

Die letzte Zeile zeigt, warum Go kein Ersatz für C ist: Es hat eine Speicherbereinigung, und damit fällt es für harte Zeitschranken aus — die Rechnung aus Kapitel 1. Band 22 behandelt, wofür es dafür die richtige Wahl ist.

Die Bestandsfrage ehrlich rechnen

Weg	Aufwand	Risiko	
Alles neu	**Jahre**	hoch	**Band 17**
Neues in anderer Sprache	gering	gering	**empfohlen**
Grenzen zuerst	mittel	gering	
Bei C bleiben	keiner	bleibt	

Die zweite Zeile ist derselbe Vorschlag wie in Band 19, Kapitel 4: Neue Bestandteile in einer speichersicheren Sprache, alte unangetastet. Die Rate neuer Speicherfehler sinkt binnen weniger Jahre erheblich, ohne dass eine alte Zeile angefasst wird.

Aufgabe 14.1.

(a) Entscheiden Sie für drei Bestandteile.
(b) Rechnen Sie die Gesamtkosten aus.
(c) Bestimmen Sie, was an der Grenze zu beachten ist.

Lösung Schritt für Schritt.

(a) Drei Entscheidungen.

Bestandteil	Schranke	Speicher	Wahl
Messerfassung	2 **ms hart**	64 KB	**C oder Rust**
Übertragung	1 s weich	8 MB	**Go**
Auswertung	keine	4 GB	**Python**

Drei Bestandteile desselben Systems, drei verschiedene Antworten — und keine davon ist eine Geschmacksfrage. Die Schranke in der ersten Zeile schließt alles mit Speicherbereinigung aus; die dritte Zeile hat keine Schranke und dafür viel Fachlogik.

Das ist die eigentliche Antwort auf die Titelfrage dieses Kapitels: nicht „welche Sprache für das System“, sondern „welche für diesen Bestandteil“. Systeme sind selten in einer Sprache geschrieben, und das ist kein Mangel.

(b) Gesamtkosten.
Der Fall. Die Übertragungsschicht, 8000 Zeilen in C oder 3000 in Go.

Posten	C	Go
Erstellung	14 PM	**6** PM
Prüfung	6 PM	3 PM
Fehler im ersten Jahr	≈ 34	≈ 11
Davon Speicherfehler	≈ **19**	**0**
Pflege je Jahr	4 PM	2 PM
Über 5 Jahre	**40 PM**	**19 PM**

Faktor zwei über fünf Jahre — und die vierte Zeile trägt den größten Teil davon. Neunzehn Speicherfehler im ersten Jahr sind nicht Nachlässigkeit, sondern die Rate, die sich aus der Zahl der Gelegenheiten ergibt.

Die Rechnung kippt, sobald eine harte Zeitschranke hinzukommt — dann ist Go schlicht ungeeignet, und die Kostenfrage stellt sich nicht mehr. Deshalb steht die Eignungsfrage vor der Kostenfrage.

(c) An der Grenze.

Frage	Wer entscheidet	
Wer besitzt den Speicher?	**festlegen**	häufigster Fehler
Wer prüft die Länge?	**beide**	
Was bei Ausnahme?	**darf nicht über die Grenze**	
Wer wandelt Zeichen um?	festlegen	Kap. 4
Blockiert der Aufruf?	**entscheidend**	**Fadenmodell**

Die dritte Zeile ist eine harte Regel: Eine Ausnahme darf niemals über eine Sprachgrenze laufen — der C-Aufrufer räumt nicht auf, und der Zustand bleibt beschädigt. An der Grenze wird jede Ausnahme gefangen und in einen Fehlercode verwandelt.

Die letzte Zeile wird oft übersehen: Ein blockierender C-Aufruf hält in Go den ausführenden Systemfaden an. Bei wenigen Aufrufen ist das gleichgültig, bei vielen bricht die Nebenläufigkeit zusammen.

Die Grenze ist der teuerste Ort

Aufruf	Kosten	Bei 10^6/s
Innerhalb C	≈ 2 ns	2 ms
Go nach C	≈ 60 ns	**60 ms**
Python nach C	≈ 90 ns	90 ms

Ein Sprachwechsel kostet je Aufruf das Dreißigfache eines gewöhnlichen. Daraus folgt die Entwurfsregel: Die Grenze verläuft entlang grober *Operationen — ein Aufruf, der tausend Messwerte verarbeitet, nicht tausend Aufrufe für je einen.*

Wer eine feingranulare Schnittstelle über die Grenze legt, verliert den Geschwindigkeitsvorteil, für den er C gewählt hat — und hat die Nachteile beider Sprachen.

Chisnall[4] stellt die Frage, ob C das leistet, was ihm zugeschrieben wird; Stevens und Rago[24] zeigen, wofür die Sprache im Systembereich unverzichtbar bleibt; Love[19] behandelt dieselbe Ebene aus praktischer Sicht.

15 Der Stand im Jahr 2026

Was gilt und was behauptet wird

Aussage	Beleglage	Urteil
Speicherfehler dominieren	eindeutig	60–70 %
Werkzeuge finden sie	eindeutig	**unstrittig**
Anordnung schlägt Verfahren	gut	Kap. 2
Undefiniertes Verhalten wird ausgenutzt	gut	zunehmend
Feste Größen sind vorhersagbar	gut	
„C ist portabler Assembler“	überholt	**Kap. 9**
„Sorgfalt genügt“	negativ	**Zahl der Gelegenheiten**
„Schneller als alles“	unvollständig	erst messen

Die zweite Zeile ist die praktisch wichtigste des ganzen Bandes. Alles, was C an Fehlern zulässt, wird von Werkzeugen gefunden, die seit Jahren verfügbar und billig sind — sie laufen nur nicht.

Fünf Entwicklungen

Erstens: Regulierung greift nach der Sprachwahl.

Vorgabe	Seit	
Empfehlungen für speichersichere Sprachen	**2023**	Behörden, USA
Fahrpläne gefordert	**2024**	**für kritische Systeme**
Produkt-anforderungen	ab 2027	EU

Erstmals wird die Sprachwahl zum Gegenstand von Vorgaben. Nicht als Verbot — gefordert werden Pläne, wie die Speicherfehlerklasse langfristig geschlossen wird. Für ein Medizinprodukt heißt das: Die Frage aus Kapitel 14 muss beantwortet und dokumentiert werden.

Zweitens: Der Standard bewegt sich.

Neuerung	Seit	
`[[nodiscard]]`	C23	**Kap. 5**
`typeof`	C23	sicherere Makros
`memset_explicit`	C23	Kap. 9
`constexpr`	C23	
Prüfbare Grenzen	in Arbeit	**Erweiterungen**

Alle Neuerungen zielen in dieselbe Richtung: dem Übersetzer mehr Prüfmöglichkeiten geben. Die letzte Zeile ist die interessanteste — Zeiger mit mitgeführter Länge, also die Ausschnittsdarstellung aus Kapitel 4 als Sprachmittel.

Drittens: Werkzeuge werden Voraussetzung.

Praxis	Verbreitung	
Sanitizer in der Prüfstrecke	üblich	
Zufallsprüfung	wachsend	**bei Bibliotheken üblich**
Reproduzierbares Bauen	wachsend	Band 19
Stückliste	gefordert	regulatorisch

Viertens: Nebenläufigkeit ist Normalfall.

> *Seit C11 hat die Sprache ein Speichermodell — vorher war jede nebenläufige Programmierung in C streng genommen außerhalb des Standards. Das ist ein leiser, aber grundlegender Fortschritt: Aussagen über nebenläufigen Code sind seither überhaupt erst möglich.*

Fünftens: Werkzeuge, die C schreiben.

Wirkung	Befund	
Erster Entwurf	schneller	
Speicherfehler	**unverändert häufig**	**Muster aus Altcode**
Undefiniertes Verhalten	häufig	
Prüfbedarf	steigt	

> *Die zweite Zeile ist die unbequeme: Vorschlagende Werkzeuge lernen aus vorhandenem C — und das enthält die Fehlerklassen dieses Bandes in großer Zahl. Der erzeugte Code sieht richtig aus und enthält dieselben Fallen. Umso wichtiger sind die Prüfer aus Kapitel 10.*

Drei Einsichten

Erstens: Die Kontrolle ist echt und sie kostet.

Größe	Gewinn	Preis
Vorhersagbarkeit	**voll**	—
Speicherbedarf	3,5× kleiner	—
Durchlaufzeit	40×	—
Zeilen	—	**14×**
Fehlerklassen	—	**fünf zusätzlich**

Beide Spalten sind wahr. Wer nur die obere sieht, wählt C zu oft; wer nur die untere sieht, zu selten. Die Entscheidung verlangt beide — und die Kenntnis, welche Größe in der eigenen Lage bindet.

Zweitens: Der Entwurf schlägt die Sorgfalt — hier besonders.

Bauform	Schließt aus	Kap.
Arena	**Speicherleck**	2
Index statt Zeiger	**Zugriff nach Freigabe**	3
Ausschnitt mit Länge	**Grenzüberschreitung**	4
Ein Aufräumpfad	**vergessene Freigabe**	5
Feste Größe	**Speichermangel**	13

Fünf Bauformen, fünf ausgeschlossene Fehlerklassen — keine davon durch Aufmerksamkeit, alle durch Struktur. Das ist derselbe Leitgedanke wie in Band 19: den Fehler unmöglich machen statt ihn zu vermeiden.

Drittens: Das Handwerk trägt über die Sprachen.

Aus Band 20	Gilt hier?	
Aufgabe verstehen	**unverändert**	
Daten zuerst	**verstärkt**	**Anordnung zählt**
Fehlersuche als Verfahren	unverändert	**plus Sanitizer**
Fremden Code lesen	unverändert	
Wann aufhören	**plus zwei Kriterien**	

Fünf Kapitel aus Band 20 gelten unverändert oder verstärkt. Was sich ändert, ist nicht das Handwerk, sondern die Zahl der Fragen, die man selbst beantworten muss — und die Werkzeuge, die einem dabei helfen.

Die ehrliche Bilanz

Frage	Antwort	
Ist C überholt?	**nein**	**vier Gründe tragen**
Ist es die Standardwahl?	**nicht mehr**	
Sollte man es lernen?	**ja**	**es erklärt alles darunter**
Neues Vorhaben in C?	begründen	Kapitel 14

Die dritte Zeile ist der Grund für diesen Band, unabhängig davon, ob man C benutzt. Wer weiß, was `append` kostet, was eine Zwischenspeicherzeile ist und warum eine Prüfung verschwinden kann, schreibt auch in anderen Sprachen besseren Code.

C zeigt Ihnen, was eine Zeile Python kostet.
Das ist sein bleibender Wert —
auch für die, die nie eine Zeile C schreiben werden.

— die Bilanz dieses Bandes.
Die Kontrolle bekommt man umsonst.
Bezahlt wird die Verantwortung, die mit ihr kommt.

Der Standard[12] legt fest, was gilt; Thompson[25] erinnert daran, wo das Vertrauen endet; Dijkstra[5] formulierte 1968 die Regel, an der sich Ablaufsteuerung bis heute misst.

Part VI

Anhang

A Regeln und Kenngrößen

Die Regeln

Zeiger	Nie ohne Länge herumreichen; Besitz aufschreiben
Zeichenketten	Ausschnitt mit Länge statt Nullterminierung
Ganzzahlen	**Vorzeichenlos nur für Größen und Bitmuster**
Messwerte	Ganzzahl mit Exponent, kein `double`
Fehler	Ein Aufräumpfad je Funktion
Speicher	**Was zusammen gelesen wird, liegt zusammen**
Belegung	Einmal für viele statt einmal je Element
Warnungen	`-Wall -Wextra -Werror` von Anfang an
Prüfstrecke	**Sanitizer bei jeder Übergabe**
Größenordnung	Vor der Wahl des Verfahrens kennen

Die wichtigsten Zahlen

Größe	Wert	Kapitel
Zugriff L1 gegen Hauptspeicher	Faktor ≈ 250	2
Zwischenspeicherzeile	64 Byte	2
Feld gegen verkettete Liste	Faktor ≈ 70	2
Belegung auf der Halde	≈ 60 ns	2
Arena statt `malloc`	Faktor ≈ 20	2
Speicher C gegen Python	Faktor $\approx 3{,}5$	1
Zeilen C gegen Python	Faktor ≈ 14	1
Lineares Suchen lohnt bis	≈ 50 Einträge	6
Füllgrad Streutabelle	max. 70 %	6
Freigabezeilen bei n Mitteln	$n(n+1)/2$	5
UBSan Laufzeitaufschlag	≈ 20 %	7
ASan Laufzeitaufschlag	≈ 100 %	7
Modulo gegen Maske	Faktor ≈ 20	13
Sprachgrenze je Aufruf	≈ 60–90 ns	14
Speicherfehler an schweren Lücken	60–70 %	15

Alle Werte sind Größenordnungen für heutige Rechner. Die Verhältnisse sind belastbarer als die absoluten Zahlen — und auf Verhältnissen beruhen die Argumente dieses Bandes.

Bauformen, die Fehlerklassen ausschließen

Bauform	Schließt aus	Kap.
Arena	**Speicherleck**	2
Index statt Zeiger	**Zugriff nach Freigabe**	3
Ausschnitt mit Länge	**Grenzüber-schreitung**	4
Verbund statt Zeiger	Besitzfrage	3
Ein Aufräumpfad	**vergessene Freigabe**	5
Aufbaupuffer	stilles Abschneiden	4
Feste Größe	**Speichermangel**	13
Zwei Zähler statt drei	Widerspruch	13
Unvollständiger Typ	Zugriff von außen	11

Neun Bauformen, neun ausgeschlossene Fehlerklassen — keine davon durch Aufmerksamkeit, alle durch Struktur. Das ist die Zusammenfassung dieses Bandes auf einer Seite.

B Landkarte und Glossar

Wo was steht

Kap.	Thema	Verweise
1	Was verborgen war	**Band 20, Kap. 1**
2	Speicher	**Band 4, Band 1**
3	Zeiger und Besitz	Band 20, Kap. 6
4	Zeichenketten	**Band 19, Kap. 5**
5	Fehler ohne Ausnahmen	Band 20, Kap. 11
6	Datenstrukturen	**Band 1, Band 20, Kap. 2**
7	Undefiniertes Verhalten	**Band 2**
8	Zahlen	**Band 0, Kap. 3**
9	Der Übersetzer	**Band 6, Band 4**
10	Werkzeuge	**Band 17, Kap. 8**
11	Module	**Band 17, Kap. 4**
12	Fehlersuche	**Band 20, Kap. 7**
13	Der zweite Entwurf	Band 20, Kap. 13
14	Wann C	Band 22
15	Stand 2026	Band 19

Begriffe

Arena	Großer Block, aus dem Stücke ohne Einzelfreigabe stammen
Ausrichtung	Adressbedingung eines Typs; erzeugt Auffüllung
Ausschnitt	Zeiger mit Länge, ohne Kopie
Besitz	Vereinbarung, wer freigibt
Halde	Speicher mit freier Lebensdauer
Keller	Speicher mit Funktionslebensdauer
Sanitizer	Prüfer, der zur Laufzeit Verstöße meldet
Umlauf	Definiertes Überlaufverhalten vorzeichenloser Typen
Undefiniertes Verhalten	Lage, für die der Standard keine Anforderung stellt
Unvollständiger Typ	Typ, dessen Aufbau nur intern bekannt ist
Zufallsprüfung	Erzeugte Eingaben, gesteuert von der Abdeckung
Zwischenspeicherzeile	Kleinste Einheit, die der Prozessor holt

C Kommentierte Quellen

[1] Aho, A., Lam, M., Sethi, R., Ullman, J.: *Compilers: Principles, Techniques, and Tools*. 2. Aufl., Pearson 2006.

Das Standardwerk zum Übersetzerbau. Für diesen Band relevant sind die Kapitel zur Zwischendarstellung und Optimierung — sie erklären, warum der Übersetzer tut, was er tut.

[2] Bentley, J.: *Programming Pearls*. 2. Aufl., Addison-Wesley 1999.

Führt vor, wie die Wahl der Anordnung ein Programm um Größenordnungen ändert. Die Beispiele sind in C, und das ist kein Zufall: Nur dort sieht man die Anordnung überhaupt.

[3] Bryant, R., O'Hallaron, D.: *Computer Systems: A Programmer's Perspective*. 3. Aufl., Pearson 2015.

Das beste Buch darüber, was unter dem Code geschieht. *Speicherhierarchie, Verknüpfung, Ausnahmen, Nebenläufigkeit — alles aus Sicht des Programmierers statt des Hardwareentwicklers.*

[4] Chisnall, D.: *C Is Not a Low-Level Language*. ACM Queue 16(2), 2018.

Der Aufsatz, der mit einer Gewissheit aufräumt. *Zeigt, dass das C-Maschinenmodell einem Prozessor von 1978 entspricht und heutige Hardware erhebliche Umwege geht, um es aufrechtzuerhalten. Zehn Seiten, frei verfügbar.*

[5] Dijkstra, E.: *Go To Statement Considered Harmful*. CACM 11(3), 1968.

Anderthalb Seiten, die eine jahrzehntelange Auseinandersetzung auslösten. Lesenswert vor allem, um zu sehen, wie eng das Argument tatsächlich gefasst war — es trifft das Aufräummuster aus Kapitel 5 nicht.

[6] Drepper, U.: *What Every Programmer Should Know About Memory*. 2007.

Hundertvierzig Seiten über die Speicherhierarchie*, gründlicher als jedes Lehrbuch. In Einzelheiten veraltet, in den Größenordnungen und Prinzipien unverändert gültig. Frei verfügbar.*

[7] Fog, A.: *Optimization Manuals*. Fortlaufend.

Gemessene Befehlszeiten und Optimierungshinweise für heutige Prozessoren. Kein Buch zum Lesen, sondern zum Nachschlagen, wenn man wissen muss, was ein Befehl tatsächlich kostet.

[8] Goldberg, D.: *What Every Computer Scientist Should Know About Floating-Point Arithmetic*. ACM Computing Surveys 23(1), 1991.

Der Aufsatz zum Thema. *Nach der Lektüre verwendet man Gleitkomma nicht mehr falsch — und weiß, wann man es nicht verwenden sollte. Anspruchsvoll und die Mühe wert.*

[9] Gustedt, J.: *Modern C*. 2. Aufl., Manning 2019.

Zeigt C, wie es heute aussehen sollte, statt im Stil von 1988. Behandelt das Speichermodell, atomare Typen und die neueren Sprachmittel. Frei verfügbar.

[10] Hanson, D.: *C Interfaces and Implementations*. Addison-Wesley 1996.

Das beste Buch über Schnittstellenentwurf in C. *Vierundzwanzig durchgearbeitete Module mit Vertrag, Umsetzung und Begründung. Alt, unverändert vorbildlich.*

[11] Hoare, C. A. R.: *Null References: The Billion Dollar Mistake*. Vortrag, 2009.

Hoare nennt die Einführung des Nullzeigers 1965 im Rückblick seinen teuersten Fehler. Ein Fachmann, der öffentlich eine Entscheidung bereut — selten und lehrreich.

[12] ISO/IEC 9899: *Programming languages — C*. C17, C23.

Der Standard selbst. Trocken und die einzige verbindliche Quelle für die Frage, ob etwas erlaubt ist. Der Anhang mit den über zweihundert Fällen undefinierten Verhaltens lohnt einen Blick.

[13] Kernighan, B., Pike, R.: *The Practice of Programming*. Addison-Wesley 1999.

Handwerk statt Methodik: Fehlersuche, Prüfung, Werkzeuge, Portabilität — überwiegend an C-Beispielen. Zweihundert Seiten, die man in einem Wochenende liest und danach anders arbeitet.

[14] Kernighan, B., Ritchie, D.: *The C Programming Language*. 2. Aufl., Prentice Hall 1988.

Unübertroffen und wahrscheinlich unübertrefflich. *Zweihundert Seiten, die eine ganze Sprache vollständig und ohne ein überflüssiges Wort erklären. Wer C lernen will, nimmt dieses Buch — alles andere ist Ergänzung.*

[15] King, K. N.: *C Programming: A Modern Approach*. 2. Aufl., Norton 2008.

Die gründliche Alternative für den Einstieg, ausführlicher als [14] und mit Übungen. Wo Kernighan und Ritchie verdichten, erklärt King — beides hat seinen Platz.

[16] Klemens, B.: *21st Century C*. 2. Aufl., O'Reilly 2014.

Behandelt, was um die Sprache herum nötig ist — Werkzeugketten, Bibliotheken, moderne Sprachmittel. Meinungsfreudig und ein nützliches Gegengewicht zum Stil der Achtzigerjahre.

[17] Koenig, A.: *C Traps and Pitfalls*. Addison-Wesley 1989.

Sammelte die Fallen, bevor sie zu Sicherheitslücken wurden. Dünn, alt und in der Sache unverändert gültig — die meisten dieser Fallen gibt es noch.

[18] Lattner, C.: *What Every C Programmer Should Know About Undefined Behavior*. 2011.

Drei Blogbeiträge, die erklären, wie ein Übersetzer aus undefiniertem Verhalten schließt. *Von einem der Hauptentwickler geschrieben — man erfährt, wie der Optimierer denkt, nicht wie er denken sollte.*

[19] Love, R.: *Linux System Programming*. 2. Aufl., O'Reilly 2013.

Praxisnahe Darstellung der Systemschnittstelle. Die Ergänzung zu [24], wenn man weniger Vollständigkeit und mehr Anwendungsbezug braucht.

[20] MISRA: *MISRA C:2012 Guidelines*. MIRA 2012, mit Ergänzungen.

Die Kodierrichtlinie für sicherheitskritische Systeme. Streng, teilweise überstreng — und lehrreich, weil zu jeder Regel die Begründung steht. Für Medizinprodukte faktisch der Maßstab.

[21] Regehr, J.: Beiträge zu undefiniertem Verhalten. Fortlaufend.

Sammelt Fälle aus echten Systemen und untersucht sie mit Werkzeugen. Wer sehen will, wie oft undefiniertes Verhalten in gepflegtem Code steckt, findet hier die Belege.

[22] Ritchie, D.: *The Development of the C Language*. HOPL-II, 1993.

Der Rückblick des Erfinders. Erklärt, aus welchen Zwängen die Entscheidungen entstanden — einschließlich der Nullterminierung, die ein bis zwei Byte je Zeichenkette sparte.

[23] Seacord, R.: *Secure Coding in C and C++*. 2. Aufl., Addison-Wesley 2013.

Ordnet die Fehlerklassen den zugehörigen Angriffen zu und nennt zu jeder die Abhilfe. Das Buch, das die Verbindung zwischen Band 19 und diesem Band herstellt.

[24] Stevens, W. R., Rago, S.: *Advanced Programming in the UNIX Environment*. 3. Aufl., Addison-Wesley 2013.

Das Standardwerk zur Systemschnittstelle. *Prozesse, Signale, Dateien, Nebenläufigkeit — mit einer Gründlichkeit, die man sonst nirgends findet. Zum Nachschlagen unersetzlich.*

[25] Thompson, K.: *Reflections on Trusting Trust*. CACM 27(8), 1984.

Drei Seiten über die Grenze des Vertrauens: ein Übersetzer, der eine Hintertür in jeden neuen Übersetzer schreibt, ohne dass sie im Quelltext steht. In zwanzig Minuten gelesen, danach denkt man anders über Werkzeugketten.

[26] van der Linden, P.: *Expert C Programming: Deep C Secrets*. Prentice Hall 1994.

Erklärt die Eigenheiten der Sprache mit den Beispielen, an denen sie tatsächlich schiefgehen — Deklarationssyntax, Feld-Zeiger-Unterschied, Aliasfragen. Unterhaltsam geschrieben und sachlich präzise.

[27] Werkzeugdokumentation: AddressSanitizer, UndefinedBehaviorSanitizer, ThreadSanitizer, Valgrind, libFuzzer.

Frei verfügbar und kürzer, als man erwartet. Wer eine Stunde investiert, hat danach eine Prüfstrecke, die mehr findet als jede Durchsicht — Kapitel 10 rechnet es vor.

[28] Wirth, N.: *Algorithms + Data Structures = Programs*. Prentice Hall 1976.

Der Titel ist die These und enthält eine Rangordnung. In C gilt sie doppelt: Die Datenstruktur bestimmt nicht nur das Verfahren, sondern über die Speicheranordnung auch die Geschwindigkeit.

Wenn Sie nur drei Bücher kaufen

[14]	**Kernighan, Ritchie**	**Für die Sprache — unübertroffen**
[3]	**Bryant, O'Hallaron**	Für das, was darunter liegt
[10]	**Hanson**	**Für den Entwurf von Schnittstellen**

> *Kernighan und Ritchie lehren die Sprache, Bryant und O'Hallaron die Maschine, Hanson den Entwurf. Auffällig ist, was fehlt: kein Buch über Werkzeuge — deren Dokumentation ist kürzer und aktueller als jedes Buch darüber sein könnte.*
>
> *Zwei Ergänzungen: [4] Chisnall, zehn Seiten gegen eine liebgewonnene Gewissheit. Und [18] Lattner — weil man undefiniertes Verhalten erst versteht, wenn man weiß, wie der Optimierer daraus schließt.*